The Australian National University
2600 ACT | Canberra | Australia



Australian National University

School of Computing

College of Engineering, Computing and Cybernetics (CECC)

Analyzing and Exploiting Memory Hierarchy Parallelism with MLP Stacks

— 12 pt research project (S1/S2 2024)

A report submitted for the course
COMP3770, Computing Research Project (R&D)

By:
Adnan Hasnat

Supervisors:
Dr. Shoaib Akram
Dr. Wim Heirman

November 2024

Declaration:

I declare that this work:

- upholds the principles of academic integrity, as defined in the Academic Integrity Rule;
- is original, except where collaboration (for example group work) has been authorised in writing by the course convener in the class summary and/or LMS course site;
- is produced for the purposes of this assessment task and has not been submitted for assessment in any other context, except where authorised in writing by the course convener;
- gives appropriate acknowledgement of the ideas, scholarship and intellectual property of others insofar as these have been used;
- in no part involves copying, cheating, collusion, fabrication, plagiarism or recycling.

I acknowledge that I am expected to have undertaken Academic Integrity training through the Epigeum Academic Integrity modules prior to submitting an assessment, and so acknowledge that ignorance of the rules around academic integrity cannot be an excuse for any breach.

November (2024), Adnan Hasnat

Abstract

Scaling performance in modern high performance computing requires exploiting as much memory bandwidth as possible. Memory-level parallelism (MLP), measuring the average number of accesses to DRAM per cycle with at least one access, is a metric related to throughput used to analyse these bottlenecks. While extensive study has been done using MLP and studying DRAM bottlenecks, there are four main gaps in existing literature: (1) connecting MLP to overall performance, (2) understanding the role of caches and cache-level parallelism, (3) quantifying the impact of prefetchers and (4) quantifying the bottlenecks on MLP. We propose a new taxonomy of MLP metrics including splitting MLP into core and prefetch components, and measuring both miss and hit cache level parallelism (MCLP and HCLP). Additionally, we present MLP stacks, a visualisation that integrates these metrics while also reporting the CPI contribution (relative effect on execution time) of each memory level to connect the MLP back to performance. The stack also shows speculative parallelism from dependency bound and structural-hazard bound loads, indicating a direction to target optimisations to improve MLP. We implement these stacks in the processor simulator Sniper and provide a few case studies on using the metric on benchmarks such as GAP. We explore the potential both for targeting software optimisations such as software prefetches, and for targeting hardware improvements such as increasing load queue sizes, towards the specific bottlenecks revealed by the stack and how this can allow more targeted optimisation.

Table of Contents

1 Introduction and Motivation **1**

2 Background **5**
2.1 Out-of-order execution and modern processor architecture 5
2.2 Cache hierarchy 6
2.2.1 Banking and non-blocking cache architecture 8
2.3 Stack visualisations 10

3 Related Work **13**
3.1 Chou, 2004 (Microarchitectural Optimisations and MLP) 13
3.2 Mehta, 2022 (Performance Analysis with Little's Law) 15
3.3 Tang, 2019 (Co-optimising CLP and MLP) 17
3.4 Carlson, 2015 (Epoch Profiles) 18

4 Design and Implementation **21**
4.1 Defining new MLP metrics 21
4.1.1 Traditional MLP 21
4.1.2 Extending MLP for prefetchers 22
4.1.3 Extending MLP for caches 23
4.1.4 Speculative MLP 24
4.2 Implementation in Sniper 24
4.2.1 Accounting core accesses 25
4.2.2 Accounting prefetches 26
4.2.3 Speculative MLP 27
4.3 MLP visualisation design 27

5 Experimental Methodology **31**
5.1 Default hardware 31
5.2 Benchmarks 31
5.2.1 SPMV 32
5.2.2 GAPBS 34

6 Results **35**
6.1 SPMV . . . 35
6.2 GAPBS . . . 38
6.2.1 Initial exploration . . . 38
6.2.2 Using speculative MLP . . . 45

7 Threats to Validity **49**
7.1 Accounting error . . . 49
7.2 Runtime of GAPBS kernels . . . 50

8 Concluding Remarks **51**
8.1 Limitations of our design . . . 52
8.2 Future work . . . 54

Bibliography **55**

Chapter 1

Introduction and Motivation

It is commonly known that retrieving and managing data is one of the main bottlenecks in high performance computing. Small amounts of data can be stored in fast on-chip Static Random Access Memory (SRAM), but SRAM is very expensive per byte of capacity and is not feasible for storing the gigabytes of data which many modern programs work with during live execution. Most data must be stored in high capacity Dynamic Random Access Memory (DRAM) chips that lie outside of the main processor. Improvements in DRAM technology have long been unable to keep up with processor improvements, partly due to the bottleneck posed by the physical distance between processor and DRAM. This causes a memory wall where getting the data for a computation from memory (loading) or writing the results of computation to memory (storing) is much slower than doing the computation itself. Many methods exist to mitigate this. The two primary strategies are:

1. Caching, where upon the processor requesting data one retrieves not just the data requested but also nearby data from DRAM, returning the requested data to the processor while storing the nearby data in fast SRAM cache memories. The idea is then that the next few requests are likely to be nearby data and can then be served directly from caches

2. Out-of-order (OOO) execution, where the processor examines a window of future instructions. Thus when a long-latency DRAM request occurs, it is able to schedule other computation while waiting, thus hiding the latency by still making use of that time.

However, accesses that can't be served from caches, i.e. cache misses, still incur high latency, and the OOO scheduler will eventually run out of alternative computation to schedule.

One further strategy is to maximise Memory Level Parallelism (MLP) by sending as

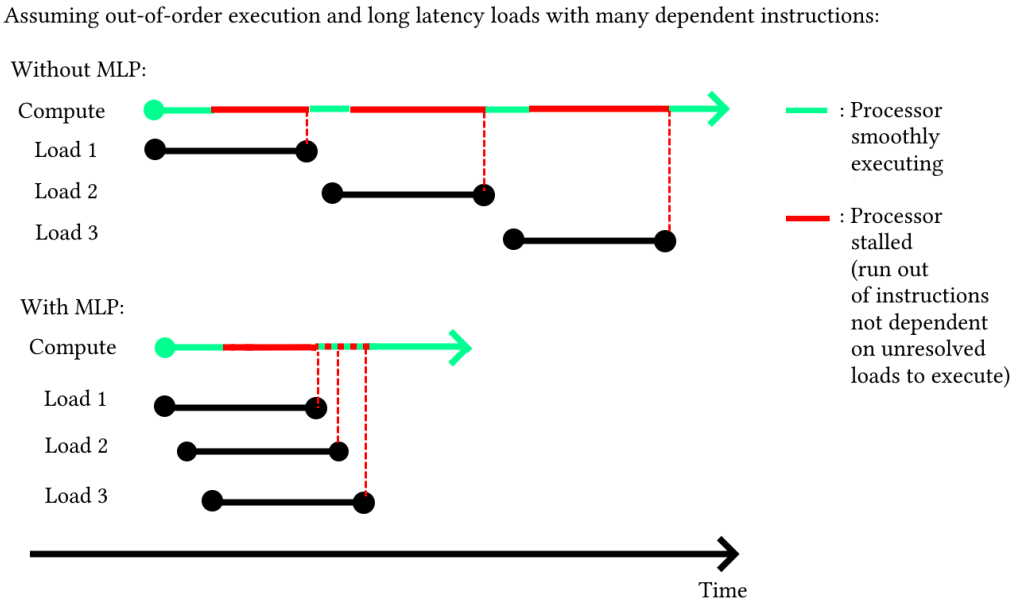


Figure 1.1: Figure showing hypothetical example of how MLP improves performance. The loads are assumed to miss caches and have many dependent instructions following them. Processor is able to keep executing for a little while after issuing each long latency load, but eventually runs out of independent instructions to issue. In the above case, the processor repeatedly stalls on each of the loads. In the below case, the stall period for the three loads is partially overlapped, reducing the total stall time and allowing much faster execution.

many DRAM accesses in parallel as possible. DRAM is often designed to be able to serve many requests in parallel, e.g. by being divided into several banks that can retrieve data concurrently. By exploiting this, the processor can overlap the latency of several loads and spend less time waiting in total, as illustrated in Figure 1.1. Simply having the core send more memory requests by either more aggressive parallelisation (hyperthreading/multicore) or larger instruction windows for finding independent loads to issue, i.e. targeting Instruction Level Parallelism (ILP), automatically improves MLP. However, there are also other ways such as using cache hardware prefetchers that preemptively request from DRAM that it believes the processor will need, injecting more memory requests to make up for the core not being able to extract MLP on its own. MLP is an important consideration for performance which both hardware architects (Chou et al. (2004); Carlson et al. (2015a)) and software designers (Mehta (2022); Zeitak and Morrison (2021)) have worked on optimising in the past.

There are, however, gaps in the literature regarding tools to analyse and find bottlenecks in MLP. MLP is typically quantified as a metric, given as the average number of useful

accesses to DRAM across cycles where there is at least one access to DRAM, which can be used to evaluate and compare different strategies for improving MLP. Simply measuring this metric however does not provide the full picture. There are other aspects which one are interested in, studying which forms the core motivations of this paper:

1. **Motivation 1:** Whether improving MLP actually improves the performance of the program. Because caches and out-of-order execution can already hide the latency of accessing memory, improving MLP only actually has a benefit if the system is struggling to hide the latency of cache misses and must adopt further strategies to mitigate it.

2. **Motvation 2:** The role of caches and cache-level parallelism in the performance of the program. Caches can factor here in two different ways.

    - **Motivation 2a**: The first is the parallelism of cache *misses*. All memory accesses which reach DRAM had to miss all of the prior cache levels. However, non-blocking caches are only able to support as many concurrent misses as the number of miss status handling registers (MSHRs) they contain, meaning that if the MSHR capacity is reached, further memory accesses cannot proceed and further MLP cannot be achieved. Mehta (2022) demonstrates that understanding this bottleneck is helpful, as they demonstrate software optimisations can be targeted by choosing optimisations that increase MLP only if the MSHR capacity would support it and otherwise choosing optimisations targeting cache miss rate.

    - **Motivation 2b**: The second is the parallelism of cache *hits*. In modern multicore architectures, accesses to caches which are shared between cores can be high latency operations even if they are hits. As described in Tang et al. (2019), this is because of more complicated interconnects for routing accesses to caches causing network contention. Hence, in addition to overlapping DRAM access latencies, one may be interested in also overlapping shared cache access latencies.

3. **Motivation 3**: Understanding core and prefetcher accesses and their impact on MLP separately, Prefetchers can make up for the fact that the core is unable to extract MLP on its own by injecting extra accesses. We would like to quantify exactly how much extra MLP this adds, and how helpful it is.

4. **Motivation 4**: Providing an indication of what are the bottlenecks on MLP. The ideal would be a quantitative approach where each bottleneck can be attributed some factor corresponding to how important it is, e.g. how much extra MLP might be achieved if the bottleneck is removed. This would then allow targeting optimisations based on seeing what the most important bottleneck to target is. Motivation 2a partly includes this motivation as it involves displaying the effect of the MSHR as a bottleneck on MLP.

We present a proposal for a visualisation that integrates several parameters connected to

MLP for the purpose of better understanding the MLP of a program, which we term the MLP stack. In addition to showing DRAM level MLP, we quantify and show cache-level parallelism split into miss cache level parallelism (MCLP) and hit cache level parallelism (HCLP), developing a taxonomy for studying the two types of cache-level behaviours discussed prior. It also shows the amount of execution time spent waiting for accesses to each cache level and DRAM, which elucidates how well the CPU is able to hide memory latency using OOO execution and how much latency from memory accesses actually affects processor execution. Finally, it shows an estimate of how much MLP could be improved if the OOO processor was able to resolve hazards inhibiting the execution of further loads.

We implement this visualisation in the Sniper multicore simulator and explore a couple of case studies on real programs. We make full use of the simulator's ability to provide a detailed view of the processor's execution in real time and hence avoid the theoretical assumptions made by Mehta (2022), thus improving on past work. Our case studies also focus both on software and hardware optimisations and the ability of using the MLP visualisation to inform and understand their impact, hoping to demonstrate the tool's effectiveness for both computer architects and software designers.

Chapter 2

Background

We first provide some relevant computer architecture background for understanding the remainder of the paper. We defer detailed discussion of the quantification of MLP to Chapter 4, but the following content will connect to MLP as a general concept.

2.1 Out-of-order execution and modern processor architecture

We require understanding out-of-order (OOO) execution in some more detail as it is a primary mechanism for providing instruction-level parallelism (ILP), which is connected to improving MLP. From the programmer's perspective, a computer executes instructions in the same order as written in the program unless if there is a branch/jump instruction. However in-order execution causes the CPU to stall on every long-latency operation, as it cannot proceed to the next instruction until it has completed the current one. For MLP, since accesses to DRAM are long latency, this would restrict MLP to 1 in most cases. The process of pipelining, beginning to execute the next instruction while the current one is executing with the idea that the two instructions will always be using different parts of the processor and thus avoid conflicting, can rectify this somewhat. Chou et al. (2004) also discusses some other ways MLP greater than 1 can still be achieved in an in-order processor. However, the program order still remains a significant bottleneck.

The key insight of OOO is that while the program order induces data dependencies between instructions that provide constraints as to how they can execute, e.g. instruction i1 may compute $A = B + C$ and the subsequent instruction i2 may compute $D = 2 * A$, in which case i2 cannot be done before i1, *independent* instructions following the current instruction can still theoretically be executed now. The processor is hence not completely bound by the program order and can re-order it dynamically. Another common

technique which will be relevant to us is *speculation*, which is bypassing a dependency bottleneck by preemptively guessing the result of an instruction, speculatively executing future instructions based on this guess, and successfully bypassing the bottleneck if the guess is correct while retaining a mechanism for rolling back the speculatively executed instructions if the guess was wrong. Conditional branch prediction is the most common version and basically involves guessing the condition clause of "if" statements. Load speculation is also a technique relevant to us; when issuing a load instruction out of order, store instructions previous to that load whose address has not yet been computed may in fact store to the same address, which would cause a data dependency. We overcome this by speculatively executing the load anyway and rolling back if an address conflict is detected later.

There are several data structures used by a speculative OOO processor which are of interest to us. The two main OOO data structures are the Reorder Buffer (ROB) and Issue Queue (IQ), with the IQ also sometimes being called Reservation Stations (RS). The reorder buffer is a circular buffer that holds all of the instructions currently within the processor backend *in order*, which is used to figure out what instructions may have been affected by a mis-speculation and roll back. The issue queue holds instructions as they come through the front end and tracks whether instructions are ready to begin executing (i.e. their operands have been calculated), which is used by the scheduler to pick instructions to issue. The ROB and IQ have fixed sizes which represent how far ahead in the program (the "instruction window") the OOO processor is able to look for independent instructions, which is a key limiting factor to how much ILP and MLP can be achieved. Load speculation also requires having a load queue (LQ) and store queue (SQ) for tracking loads and stores, whose sizes also limits execution of loads and MLP. Figure 2.1 shows a schematic including some of these data structures and more.

These data structures require facilitating using fast SRAM memory in order to be viable, but SRAM is expensive. At the same time, there are many different data structures and sizes to consider, and an architect must provision their SRAM budget across these structures. This leads to many difficult decisions and tradeoffs when provisioning sizes, navigating which is a key aspect of the hardware architect's challenge. We will study ROB/IQ/LQ/SQ sizing decisions and their effect on MLP as part of our investigation in order to address this problem.

It is also worth noting that in a *superscalar* processor, the multiple instructions are dispatched and issued per cycle, which often relies on duplicated functional units that can be issued to separately. This further improves ILP and MLP.

2.2 Cache hierarchy

We provide some background on the working of caches, which is relevant to our discussion of cache-level parallelism. The modern cache hierarchy tends to consist of up to 4 levels named L1 to L4, with the last of these levels sometimes also called the last level cache

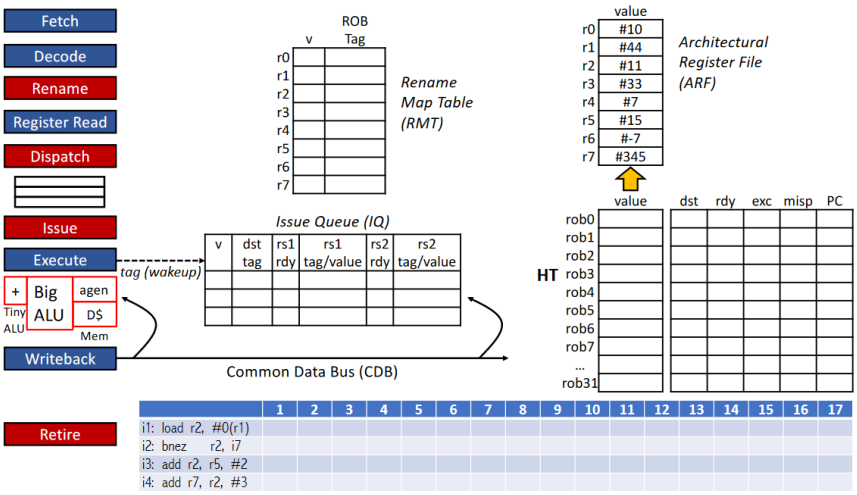


Figure 2.1: Example schematic of a (superscalar) OOO processor. The left side shows the processor pipeline stages, showing the execution flow of an instruction. The remaining parts of the diagram show the ROB and IQ as well as other data structures. Image credit to Shoaib Akram, from slides for COMP3710 Microarchitecture 2022.

(LLC). Each cache contains more space than the next one, and a load travels down the order of caches while looking for the requisite data, i.e. if it is not in L1 check L2, and if it is not there check L3, and so on. Memory is typically byte addressed and data is requested by the processor in units of *words*, which corresponds to the size of registers (the units of SRAM memory inside the core processor which are directly used by computation as local scratchpads) in the processor. Caches however request a larger unit, the cache block size, from DRAM which contains nearby data, with the cache space being split into blocks correspondingly (also sometimes called cache lines). If level L is hit by a load, the cache block accessed is copied into all levels above, so that if the next access is to data within the same cache block, it will hit immediately in L1 and resolve very quickly. Data brought into a cache replaces other data, which is evicted. The decision as to what data to evict depends on the cache policy, but generally older or less used data is evicted. How frequently data is evicted depends on the cache capacity, so at L1 it is evicted very quickly and requires subsequent accesses to be within a short timeframe to hit, while at L2/L3 it takes longer to be evicted. The use of multiple cache levels smoothens out the eviction process and allows a range of time for accesses to cached data to make it. Figure 2.2 depicts an example cache hierarchy. We are interested in studying parallelism across the hierarchy.

How caches behave with writes/stores is more complicated but is not necessary to discuss in detail for our purposes. It is also worth noting that L1 cache is typically split between

L1-D and L1-I. Because program instructions are still data and are typically stored and fetched from memory, instruction fetch consists of a memory access as well. Instructions fetched are also cached, with the L1-I cache being exclusively for instructions. We do not consider the effect of instruction fetches in great detail in this study, but they do have an effect on MLP as we will discuss in related work when we discuss Chou et al. (2004).

In modern multicore processors, caches are further classified between private (L1 typically, L2 potentially) and shared (LLC typically). This introduces additional aspects to the cache access protocol for maintaining coherency of data between cores via snooping into sibling caches. There may also be additional off-chip levels between the LLC and DRAM, such as high bandwidth memory (HBM), which use slower technology than SRAM caches but faster than DRAM. Both of these are depicted in Figure 2.2 and introduce additional layers to consider when quantifying cache-level parallelism, but are beyond the scope of this project.

An aspect shown in the figure that *is* relevant is the presence of hardware prefetchers. Modern cache units tend to have prefetcher modules embedded. These have data structures that track the cache access pattern recently observed and try to identify patterns. In a linear stride prefetcher, this is as simple as identifying accesses which increase linearly in addresses, which tends to correspond to traversing through an array (very common operation). The prefetcher then injects additional requests to the next address in these patterns pre-emptively. Ideally the time spent resolving prefetch requests is then overlapped with compute and not seen by the processor, and following the prefetch the data is now in the cache, which the processor will hit when it actually does request that data. The prefetcher injects additional requests, improving MLP; this is the effect we are interested in studying.

2.2.1 Banking and non-blocking cache architecture

Since we are studying parallelism, it is worth discussing the mechanisms by which caches and DRAM serve requests in parallel.

DRAM is split into a hierarchical organisation; the total DRAM of a program consists of several channels, which are split into DIMM modules, which are split into ranks, which are split into chips and banks, which are finally split into rows. The sections of DRAM within each level tend to store different subsections of the data while being able to operate independently, which is the mechanism that supports parallelism. Figure 2.3 depicts chip and bank level parallelism. Chips each store a different byte of the word aligned to a word boundary, such that each byte of the word can be supplied from each chip in parallel while each chip has to do less work overall. The chips themselves are then also split into banks, such that two different accesses to words across different banks can also be served in parallel. Bank level parallelism is one of the key aspects relevant to MLP since it occurs directly at the word level, though parallelism at many other levels (DIMM level, channel level, etc) also exists (though generally requires accesses to very

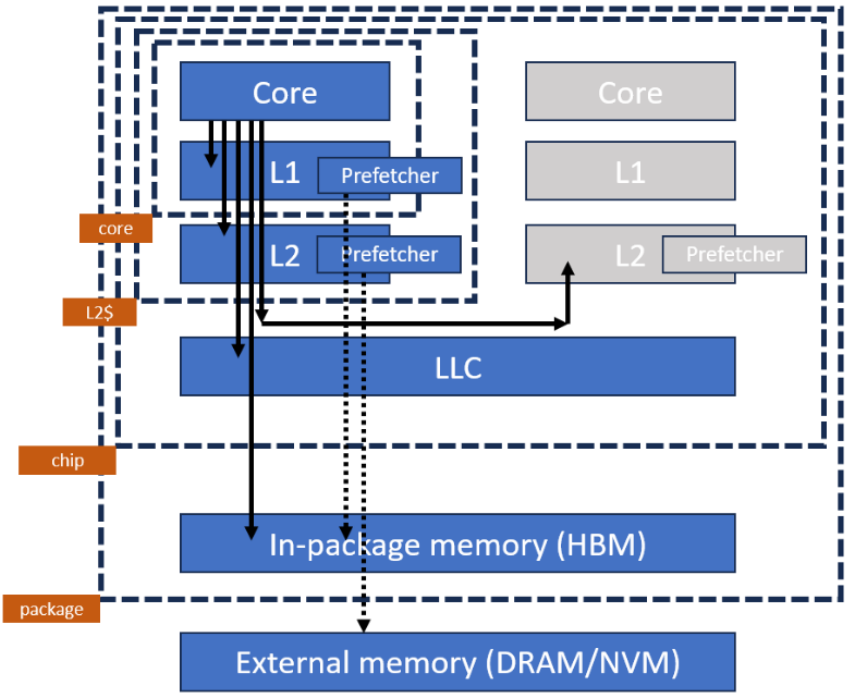


Figure 2.2: Example of a complex cache hierarchy in a modern multicore system. Shows how accesses traverse the hierarchy. Image credit to Wim Heirman.

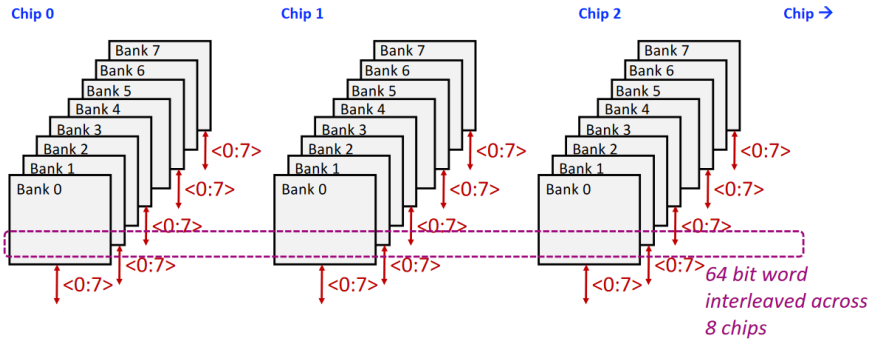


Figure 2.3: Showing how banks are organised in DRAM for serving accesses in parallel. Credit to Shoaib Akram, from slides for COMP3710 Microarchitecture 2022.

different addresses).

For cache parallelism, the first step is non-blocking cache architecture. In prototypical caches, misses and hits would be processed one at a time and subsequent accesses would have to wait. The three types of parallelism which we want to enable are serving misses in parallel (miss-under-miss), serving a hit while a miss is ongoing (hit-under-miss) and serving hits at the same time (hit-under-hit). Serving multiple misses is achieved through the miss-status handling registers (MSHRs) discussed in the introduction. This also serves to handle hit-under-miss. Serving hit-under-hit is more complicated, but inspiration is often taken from DRAM which leads to cache being split across multiple banks serving parallel accesses. An example complex cache architecture, in particular a shared LLC, is shown in Figure 2.4.

2.3 Stack visualisations

A common type of visualisation in microarchitecture which may inform our approach is the stack visualisation. The purpose of a stack is to display the peak value of a metric as well as the actual achieved value of the metric, and attribute the gap in between the two to various specific reasons by estimating how much metric was lost/gained from each reason. The primary example of this is the Cycles-Per-Instruction or CPI stack, as documented in Eyerman et al. (2006). CPI, equal to the number of cycles divided by the number of instructions, measures the time spent per instruction. The peak CPI shown in the stack is assuming no miss events, i.e. the instructions are dispatched at the peak issue width of the processor every cycle. One can then consider each type of miss event and the number of cycles per instruction lost to each, which forms components that stack on the peak CPI up to the actual achieved CPI. Miss events include waiting for memory or cache accesses (DRAM/L1/L2/L3), branch misprediction and ROB or IQ becoming full, all of which we can recognise from our discussion of OOO execution. An example CPI stack with interpretation is shown in Figure 2.5. The CPI stack will

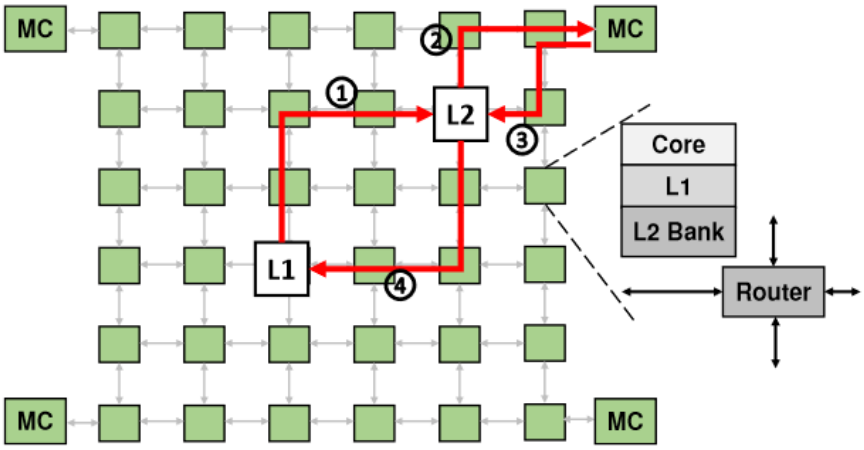


Figure 2.4: Example of a complex cache architecture for multicore. In addition to having multiple banks, the banks are distributed across nodes associated with each core and connected in a network-on-chip that forwards requests across them. Credit to Tang et al. (2019).

play a key part in our visualisation.

Other examples of stacks include the speedup stack shown in Eyerman et al. (2012), which displays the speedup achieved from applying N threads to a task and attributes the reasons why the optimal speedup of $N\times$ is not achieved, and the criticality stack shown in Bois et al. (2013), which break total execution time into "criticality" components for each thread. A similar stack-type visualisation that shows achieved MLP vs peak MLP may be appropriate in our case.

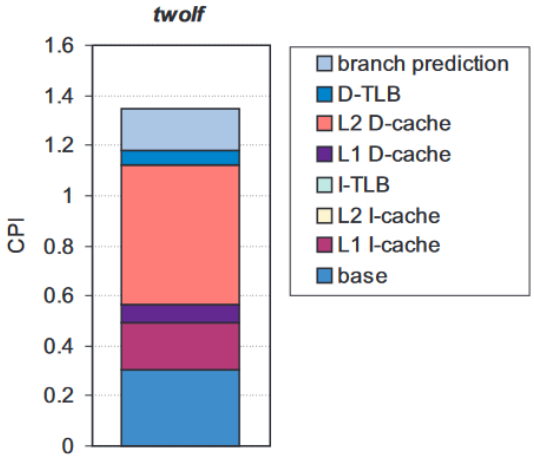


Figure 2.5: Example CPI stack from Eyerman et al. (2006). In this case, we can see that the ideal cycles per instruction from executing as many instructions as supported by the dispatch width would be 0.3 (height of the the base component), but the achieved CPI was around 1.35 (height of the top of the stack), and the primary penalty cycles came from waiting for L2 D-cache (the orange component).

Chapter 3

Related Work

Many studies have been conducted in the past on analysing DRAM level MLP, and it is a well established metric. We will go over a few of these studies and assess the methods for analysing and understanding MLP which they use, and how it compares to our contributions and motivations.

3.1 Chou, 2004 (Microarchitectural Optimisations and MLP)

Chou et al. (2004) is a relatively old work on the subject, but lays significant relevant groundwork. The paper focuses specificially on the effect of microarchitectural (i.e. hardware) optimisations on MLP rather than software, and demonstrating that targeting MLP can improve performance. They present a theoretical framework for studying MLP called the epoch model, develop a simulator based on this model which they call MLPSim, and use it to measure the effect of optimisations on both MLP and overall performance.

The epoch model presented is a qualitative method for studying MLP. It relies on the idea that the processor execution is split into time doing on-chip computation, when loads are issued in quick succession, and long periods of time spent waiting for off-chip loads to return. The periods of waiting for loads are called epochs, and the key insight is that the MLP is limited by the number of loads the processor can issue before each epoch (called the epoch set) until it is obligated to wait for these loads to complete. The model assumes that all the loads will overlap during the epoch and finish executing once it has returned. The OOO processor will examine a window of instructions beyond the first load issued (the epoch trigger), and under ideal circumstances the epoch set would include all independent loads that follow. However, Chou et al define a set of window termination conditions which cause the window to terminate early as the OOO is forced to avoid looking further forward and begin the epoch. They identify four

main termination conditions: (1) insufficient ROB/IQ size, (2) serialising instructions, (3) instruction fetch miss and (4) branch misprediction. Each of these is relatively straightforward to understand; instruction fetch miss for instance means that further instructions cannot be fetched until the miss resolves and waiting for the miss (which is a load from memory itself) begins an epoch. Microarchitectural optimisations can then be judged by their ability to address these window termination conditions. In fact, the simulator presented implementing the epoch model is able to quantify the MLP lost from each of these window termination conditions, which is used to present a stack visualisation as shown in Figure 3.1. This rather strongly addresses Motivation 4 of our study and directly provides a visual tool that competes with ours.

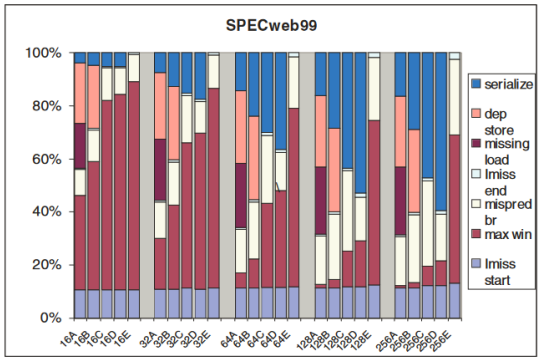


Figure 5: Factors Inhibiting Further MLP.

Figure 3.1: MLP Stack based on the Epoch Model as presented by Chou et al.

Additionally, as part of the theoretical groundwork laid by Chou et al, they also define a formula which relates overall performance to MLP, which directly addresses our Motivation 1. The formula is given in Figure 3.2. The first term in the formula, $Cycles_{perf}(1-Overlap_{CM})$, reflects the on-chip computation time (i.e. if the caches were perfect and there is no off-chip time), while the second term, $\frac{NumMisses \times MissPenalty}{MLP}$, represents the off chip computation time ($NumMisses \times MissPenalty$ is total miss penalty/latency assuming all misses are serial, which is divided by MLP to account for parallelism). This formula implies that as MLP increases, the runtime is capped at the on-chip component while off-chip component goes to 0, which is somewhat analogous to Amdahl's Law as it indicates MLP speeds up only part of a system. One obvious limitation of the formula is that it requires knowing the runtime assuming perfect cache, which is only possible in a simulator, but this can potentially be overcome by instead measuring time spent in on-chip and off-chip computation separately within a single run instead. It is also limited by the assumption that misses each take a constant amount of time. Nevertheless, it provides a simple and understandable relationship between MLP and performance. Chou et al do in fact verify the accuracy of this formula in a cycle accurate simulator (in Section 5.2 with the results shown in Table 4), and generally find agreement within 1%.

$$Cycles = Cycles_{perf}(1 - Overlap_{CM}) + \frac{NumMisses \times MissPenalty}{MLP}$$

Figure 3.2: Formula connecting MLP to overall performance. Quoting Chou et al: "$Cycles$ is total execution cycles, $Cycles_{perf}$ is number of execution cycles if the furthest on-chip cache is perfect, $Overlap_{CM}$ is the fractional overlap of compute cycles with off-chip cycles, $NumMisses$ is the number of off-chip accesses, $MissPenalty$ is the latency of each off-chip access, and MLP is the average memory-level parallelism."

Overall, Chou et al very strongly addresses several of our key motivations. Most notable is Motivation 4, as they develop a classification of MLP bottlenecks, a method of quantifying their effect and a visualisation that displays a stack indicating the relative effects of each bottleneck. They also present an estimate formula that indicates MLP reduces the off-chip access component of runtime by a factor of $1/MLP$, addressing our Motivation 1. The key areas on which we hope to improve on Chou et al are Motivations 2 and 3, i.e. isolating the effect of prefetches and including the effects of caches. Chou et al do mention both software/hardware prefetches and the fact that they increase MLP, noting that one benchmark they evaluated (SPECweb99) has higher MLP likely because it has software prefetching built in, but do not isolate or quantify the additional MLP. Regarding Motivation 1, they also do not integrate their formula for the effect of MLP into their visualisation, whereas we hope to connect performance and MLP in our visualisation directly. Nevertheless, this paper provides a good base of ideas for us to work on.

3.2 Mehta, 2022 (Performance Analysis with Little's Law)

Mehta (2022) is a more recent study. Mehta proposes a technique of measuring MLP on a real device using existing performance counters for DRAM bandwidth and latency, and formulas, most notably the titular Little's Law, which connect these to MLP. Mehta observes that MLP is bottlenecked by the MSHRQ occupancy at cache levels, and the MSHR capacity determines the theoretical peak MLP. Based on this, Mehta presents a "recipe for program optimisation" which, depending on how close the current MLP is to the theoretical peak, decides which optimisations are appropriate depending on whether they rely on increasing MLP or not. In contrast to Chou et al, Mehta focuses primarily on software optimisations such as loop reorganisation. As a sidenote, Mehta's stated motivation is much broader than only studying MLP, as they suggest that performance analysis tools and metrics in general suffer the problem of becoming too complex to interpret and action and that their recipe using MLP is simple in contrast. It is a somewhat dubious claim that MLP is indeed as simple as claimed or that it is an appropriate metric for performance analysis in all circumstances (as it is not helpful when

the program is not memory bound), but we will sidestep this aspect and focus on the MLP analysis technique demonstrated itself.

A key point which makes this study distinct from ours is the fact that it uses real hardware as opposed to a simulator. Mehta develops a formula for MLP as follows: Little's Law gives that the number of requests at any given time is the rate of arrival of requests R times the average latency lat_{avg} of each request, which gives $MLP = lat_{avg} \times R$. We then have that memory bandwidth BW is request rate times the amount of data retrieved on each request, which is the cache line size cls, so $BW = R \times cls$ and rearranging for R + substituting into the MLP formula gives the final formula:

$$MLP = \frac{BW \times lat_{avg}}{cls}$$

For each program, Mehta then uses the memory tool CrayPat which uses performance counters to compute the memory bandwidth BW achieved by that program, which is fed into a pre-computed memory bandwidth-latency plot generated empirically using the memory characterisation tool X-Mem to find lat_{avg}. Thus, they are able to compute MLP using the more easily measurable metrics of memory bandwidth and latency.

Mehta then contends that the peak theoretical MLP can be determined from the MSHR capacities. This uses the fact that the MSHR occupancies must be at least greater than the MLP observed at DRAM since the accesses which reach DRAM must have missed at all prior levels. Mehta indicates that since the L1 MSHR is much smaller than the L2 MSHR, the peak MLP is bound by L1 MSHR. This is unless if hardware prefetchers are active; Mehta seems to assume the use of conditional prefetching which activates only if the access stream is regular enough to predict. Then, with the understanding that L2 prefetchers are generally more aggressive, one can assume prefetches dominate the MLP and make L2 MSHR the bottleneck. Thus Mehta considers the L1 MSHR capacity the peak theoretical MLP if prefetchers are not active, and the L2 MSHR capacity the peak if they are.

Mehta's treatment tends to conflate MLP at DRAM level with MLP at cache levels. Using the additional terminology of cache miss level parallelism (CMLP) which we introduce, Mehta is in fact assuming that L1 CMLP is equal to the core component of MLP while L2 CMLP is equal to the core + prefetch component of MLP. This ignores the fact that there may be extra accesses in the cache hierarchy which are not seen at DRAM, though one can make the theoretical argument that such accesses are short lived in comparison to the ones which go to DRAM and hence do not contribute much average MLP. Mehta's experimental results focus on the performance improvements and changes of MLP that arise from various optimisations, and do partly validate the idea that the recipe correctly predicts the room for additional MLP through demonstrating the performance changes being consistent with the recipe. However, there are also several cases where the performance benefit is not as expected, and while Mehta outlines reasons in these cases such as resource contention or presence of another bottleneck such as "peak

achievable streams bandwidth", it is not entirely clear these reasons are valid. Mehta is partly forced to make these assumptions about CMLP because of working with just the available performance counters on a real machine, which are sufficient to reason about MLP but not CMLP; we plan to rectify this by making use of the added observability provided by a simulator.

It is also notable that this study is highly focused on software optimisations. This is on contrast to Chou et al which focused only on hardware optimisations. Mehta does evaluate the software on three different processors with different MSHRs, indicating the ability for the software optimisation strategy to work on several types of hardware, but does not view hardware optimisation itself as a goal. This also means that while Chou et al generally considered the instruction stream of the program as static and the optimal MLP to be determined by dependencies of loads in the program, thus taking the limit of MLP to be determined by the software, Mehta considers the MLP to be limited by the hardware and specifically the MSHR size, and demonstrates that the software limits considered by Chou et al can in fact be overcome by reorganising the memory access pattern itself. Our study plans to consider both software and hardware optimisations as an integrated approach.

Overall, Mehta's approach does involve considering both the effect of caches and prefetches on MLP, specifically how MSHR capacities provide a cap on cache miss parallelism and hence MLP, tying to our Motivations 2a and 3. However, Mehta only considers these effects qualitatively, and their quantitative study is limited to the final MLP observed at DRAM level and the runtime improvement achieved. We aim to introduce a taxonomy that quantifies cache and prefetch MLP, addressing this gap.

3.3 Tang, 2019 (Co-optimising CLP and MLP)

Tang et al. (2019) is another work which attempts to introduce caches into the larger picture of MLP. Tang et al observe that in addition to needing to optimise MLP in memory-bound systems, cache hits can also have high latency in highly multicore systems. Since the LLC is shared between cores, which has led to complex non-uniform cache access (NUCA) cache architectures that requires a complex on-chip network to route accesses, network contention can contribute to large latencies for accesses to LLC even when they hit. As a result, Tang et al proposes additionally defining cache hit parallelism, which is HCLP in our taxonomy but which Tang simply terms CLP, in addition to MLP. The paper focuses on software optimisations, and the primary contribution of the paper is the development of a CLP/MLP-aware compiler algorithm which uses a modified version of existing compiler techniques such as loop tiling and strip mining in addition with numerical optimisation techniques to co-optimise CLP and MLP. Our study's primary focus is measurement and visualisation, with optimisation being a side focus, and automatic optimisation to the level demonstrated by Tang et al is certainly beyond our scope. As a result, in our discussion of their work we will focus on how they quantify and measure CLP.

Interestingly, Tang et al use a different bank-aware definition of MLP which is also reflected in how they define CLP. Recall that the hierarchy of divisions of DRAM includes a division into banks, which can serve different requests in parallel and facilitate parallel handling of requests within DRAM. Similarly, LLC is also divided into banks. The idea is that while analysis of MLP when viewed from the perspective of the processor traditionally assumes that the memory system has perfect concurrency in handling accesses, there are in fact synchronisation/contention overheads present. If accesses hit the same bank, then they in fact may be serialised. Tang et al hence define MLP as the average number of DRAM banks active when there is at least one bank active, while CLP is as the average number of LLC banks active when there is at least one bank active. This definition attempts to more closely reflect the "true" parallelism of accesses to memory/cache, as the normal definition of average number of accesses are being waited for in total overestimates how many of these accesses are actually being evaluated in parallel.

Tang et al implement the measurement of MLP and CLP in a simulator, noting that existing performance counters are not sufficient to measure them on real hardware. They use the gem5 simulator system for this. However, the visualisations of MLP and CLP given are limited to simple bar graphs that just report a number for each, without any further detail. It is worth nothing that measuring the metrics themselves is actually not required for their CLP/MLP-aware compiler to work, which just requires knowing the static translation of memory addresses to banks in LLC and DRAM, and hence they are able to validate their compiler on real hardware (Intel Knight's Landing).

Overall, Tang et al is an impressive work which provides solid motivation for studying LLC hit parallelism. The development of the CLP metric addresses our Motivation 2b. The use of bank aware versions of MLP and CLP also indicates an alternative method of quantifying these which may be more useful in practice, though is complex and likely lies outside of the scope of this report to explore. However, few of our other motivations are addressed, and in general this study does not really focus on the measurement and visualisation of MLP metrics. It is worth noting that cache hit level parallelism is in fact related to cache miss parallelism (since e.g. miss + hit = total), and it may be possible to present both in a way that makes the connection between the two clear.

3.4 Carlson, 2015 (Epoch Profiles)

Regarding Motivation 1, connecting MLP to total performance, Carlson et al. (2015b) presents a visualisation which takes an inverse approach. Rather than considering MLP directly, which Carlson et al claim does not allow one to determine application performance, they consider the number of epochs per kilo instruction (EPKI). This is a quantity which is connected to MLP as it comes from the same epoch model used to define MLP in Chou et al. (2004), and which is inversely proportional to MLP, since fitting more memory accesses into a single epoch saves epochs compared to having each cause a separate epoch. At the same time, epochs directly represent time spent stuck waiting for off-chip computation and thus time lost, and hence more directly connected

to performance. EPKI also responds better to other parameters. For instance, while more cache hits leads to less accesses going out to memory and hence potentially lower MLP, it does also decrease the number of epochs and in fact we should expect an *increase* in performance, which is consistent with the change in EPKI. The Epoch Model graph then displays a heat map showing EPKI for various on-chip cache provisions (on the x axis) and various ROB/issue queue sizes (on the y axis). The heat map can then provide a tool to a hardware architect for deciding how to provision resources between cache and processor, by identifying which is more impactful.

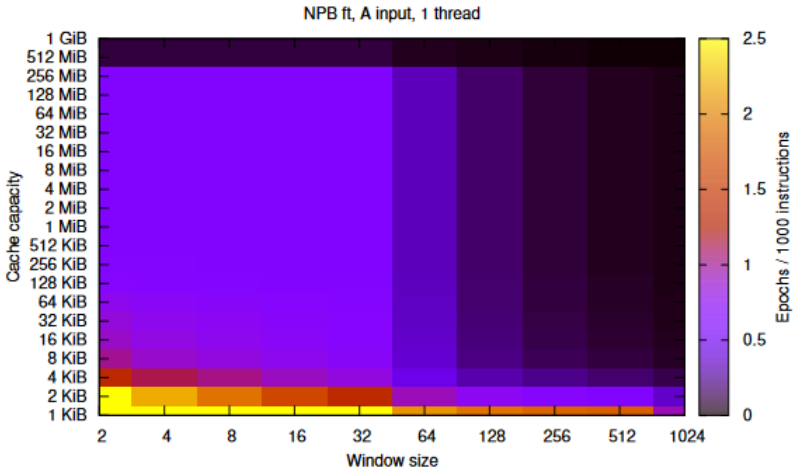


Figure 3.3: Example of an Epoch Profile, as demonstrated in Carlson et al. (2015b). Quoting Carlson et al: "This epoch profile plots the number of non-overlapped long-latency memory accesses (in epochs per 1,000 instructions) as a function of a processor's instruction window size and cache capacity. Epoch profiles show how long-latency effects impact overall performance."

This paper presents a challenge to our goal in Motivation 1, as Carlson et al suggest that MLP is inherently a difficult metric to connect to overall performance. However, we contend that the solution to this problem is to provide additional context that allows putting MLP in perspective. The formula presented in Chou et al. (2004) which we discussed earlier indicates that the off-chip time, i.e. the time spent in epochs, is in fact determined not just by MLP but also the number of misses and their average latency. This resolves the inconsistency described earlier where MLP was decreased even though performance increased; improving cache hit rate reduces the "number of misses" component of this formula. We will attempt to overcome the problem suggested by this paper by providing additional context such as this.

As for our other motivations, this paper doesn't address them much. Carlson et al. themselves acknowledge the limitation that the current Epoch Profile does not consider the effect of prefetches.

Chapter 4

Design and Implementation

We now discuss in detail the design of the MLP stack, as well as its implementation in Sniper. We recall that the four motivations of the study are to 1) identify whether improving MLP actually improves the program performance, 2) provide an understanding of cache level parallelism and 3) provide an understanding of how prefetchers affect MLP and 4) indicate information that helps in optimising MLP. Our design introduces new metrics that supplement average MLP in order to address these motivations, as well as a visualisation which puts these metrics together. We also discuss how the measurement of these metrics was implemented in the Sniper simulator.

4.1 Defining new MLP metrics

4.1.1 Traditional MLP

We first discuss, in slightly more detail, the methods of measuring parallelism of accesses at the DRAM level which form the traditional outlook of MLP. At any given time t, one can directly measure the number of parallel ongoing accesses, which gives the instantaneous MLP $MLP(t)$. Measuring $MLP(t)$ at all points in time throughout the program's execution would provide the most complete view of MLP, but this involves considering a lot of data, and one may like a more succinct measure which gives a summarised view of the average overall performance. This is the purpose of average MLP which we simply denote MLP, which is traditionally given as the average number of accesses to DRAM per cycle where there is at least one access to DRAM. This gives a formula as below, where $T(DRAM)$ is the number of cycles spent accessing DRAM:

$$MLP = \frac{\sum_t MLP(t)}{T(DRAM)} \tag{1}$$

We note here that this definition chooses deliberately to exclude considering idle cycles, i.e. when there are no accesses to DRAM, in the denominator. The rationale for this is MLP would otherwise be decreased if there is more time where DRAM is idle, but there is nothing necessarily wrong with DRAM being idle, as the processor may still get the data it requires from caches and may continue doing useful computation in this time. We are only specifically interested in the time when there is at least one access to DRAM as that is when the processor is obligated to wait for memory, i.e. is memory bound, and that is when one would like to overlap *other* accesses to DRAM over the waiting period. Note that one other consequence of this denominator is that $MLP \geq 1$ always. This denominator will be important in the following.

4.1.2 Extending MLP for prefetchers

To isolate the effect of prefetches, we would like to be able to split the MLP into the part contributed by accesses generated by the core, and the part contributed by prefetches injecting extra requests. Denoting the core component as MLP_C and the prefetch component as MLP_P, we can classify the accesses at a given time as either being from core or prefetch, and hence we define $MLP_C(t)$ and $MLP_P(t)$ as the number of core accesses to DRAM and the number of prefetch accesses to DRAM at time t respectively. This then satisfies $MLP(t) = MLP_P(t) + MLP_C(t)$, allowing us to split the MLP as desired. For the average MLP, we then have by (1):

$$\begin{aligned} MLP &= \frac{\sum_t MLP(t)}{T(DRAM)} = \frac{\sum_t MLP_C(t) + MLP_P(t)}{T(DRAM)} \\ &= \frac{\sum_t MLP_C(t)}{T(DRAM)} + \frac{\sum_t MLP_P(t)}{T(DRAM)} \end{aligned}$$

Hence, the average MLP naturally splits into two summands which we may respectively denote MLP_C and MLP_P, and define as the average core MLP and average prefetcher MLP:

$$\begin{aligned} MLP_C &= \frac{\sum_t MLP_C(t)}{T(DRAM)} \\ MLP_P &= \frac{\sum_t MLP_P(t)}{T(DRAM)} \\ MLP &= MLP_C + MLP_P \end{aligned}$$

Note that this choice entails making the denominators we average over for MLP_C and MLP_P the same, i.e. time spent having any access to DRAM, whether it be a core or prefetch. In addition to allowing us to split up the total average MLP neatly as above, this choice also makes sense because we want to overlap core accesses with *both* core and prefetch accesses, so it makes sense to consider time when either is occurring (and similarly for prefetch).

During experimentation, it emerged that while prefetches inject MLP, some prefetches prefetch data which is never used by the core and hence are useless. We can in fact split prefetch MLP further into useful prefetch MLP and useless prefetch MLP as a result, which is also integrated into the final visualisation.

4.1.3 Extending MLP for caches

We would also like to quantify parallelism of cache accesses. As discussed in our motivation, we in fact want to separately consider cache miss parallelism and cache hit parallelism, as the former determines cache MSHR contention and the latter measures overlapping of latencies of accesses to caches. We may also consider total cache access parallelism which includes both hits and misses. Defining the instantaneous versions of these metrics is relatively straightforward:

- $TCLP(Ln, t)$: total number of ongoing accesses being processed (misses or hits) at level Ln of the cache hierarchy at time t
- $MCLP(Ln, t)$: number of pending misses at level L at time t
- $HCLP(Ln, t)$: number of pending hits at level L at time t

And we may define core and prefetch components of each of these to isolate accesses attributed to core and prefetch separately as well, e.g. $HCLP_C(L3, t)$ is specifically the hits which are core accesses at level $L3$ at time t. We will use $XCLP$ to denote the specific type of CLP given by an arbitrary variable X, e.g. $X = M$ gives $MCLP$, $X = H$ gives $HCLP$, etc.

However, when computing an average for these metrics, we have some difficulties with choosing a denominator. The most obvious choice is to average each of the $XCLP_i(Ln, t)$s over the number of cycles where it is greater than 1. However, this would cause the denominators applied to different levels to be different, and we would lose the following properties obeyed by the instantaneous versions of the metrics:

- Since any core access currently at level Ln must have missed and be pending as a miss at all previous levels Lk, we have $TCLP_C(Lm, t) \leq TCLP_C(Ln, t)$ and $MCLP_C(Lm, t) \leq MCLP_C(Ln, t)$ for $m > n$. Prefetch CLP does increase at any levels where new prefetches are actively injected, but decreases at any levels beyond that. This also implies $MLP(t) \leq MCLP(Ln, t)$ for all n since any access that reaches DRAM will be pending as a miss at all prior levels.
- As a result of total = hit + miss, $TCLP(Ln, t) = MCLP(Ln, t) + HCLP(Ln, t)$.

Losing these properties would make the metrics less intuitive by obscuring the expected relationships between them, and make it harder to build a stack that conveys a cohesive picture. Our decision was hence to instead have all of the averages share the same denominator, which is the total time spent having accesses to any level of the memory hierarchy, which we will denote $T(Hier)$. This resolves the issue, but also requires us to change the denominator averaged over for DRAM MLP for the CLP and MLP to be

consistent with each other too (from $T(DRAM)$ to $T(Hier)$). This would in fact render the MLPs we may compute in our system slightly inconsistent with those reported by others, but since we are considering the memory hierarchy holistically rather than just DRAM, we considered it a reasonable decision for the denominator to include the whole memory hierarchy.

Hence, ultimately, we had:

$$XCLP(Ln) = \frac{\sum_t XCLP(Ln, t)}{T(Hier)}$$

And this involved redefining MLP at DRAM level to:

$$MLP = \frac{\sum_t MLP(t)}{T(Hier)}$$

4.1.4 Speculative MLP

A later addition to the MLP stack was the introduction of speculative MLP metrics. Following motivation 4, informing MLP optimisations, we considered the possibility for measuring an estimate for the potential for extra MLP that the program may be able to exploit. The idea we came up with is to consider the loads in the issue queue at any given time which were not able to be issued that particular cycle either due to being blocked from their address being dependent on other computations (dependency bound or DP-bound) or from the processor resources such as the load queue becoming full and the load needing to wait until resources are available (structural (hazard) bound or ST-bound). By estimating the amount of extra MLP these loads would be able to provide, one gains information regarding what type of optimisations may help.

We hence define average DP-bound and ST-bound speculative MLP as the average number of loads that are in the issue queue but either DP or ST bound, per cycle where there is at least one access to the memory hierarchy. The denominator here is somewhat arbitrary, but is kept consistent with the denominators applied elsewhere.

4.2 Implementation in Sniper

Having discussed the definition of new MLP metrics, we will now turn to discussing how they were measured in the Sniper simulator.

Our use of Sniper relied on the fact that the simulator itself is easy to modify, as the C++ codebase is highly modular and each part of the simulator has its own files that can be adjusted. Starting with Sniper version 8.0, we modified the simulator to add extra counters to measure MLP-related statistics, which were raw integer values designed generally to be quick to compute in order to optimise simulation times. We then added Python scripts that apply post-processing to turn the raw counters into MLP values to

report. The modified version of Sniper is included in the artifact submitted with this report.

Sniper has several simulation models that can be used, which we had to choose between. The interval simulation model, which is Sniper's main model as described in Carlson et al. (2011), is fast and relies mainly on considering major long-latency miss events like cache misses and the intervals of time between them. The processor is assumed to be dispatching instructions constantly and at full capacity most of the time, and is only interrupted if significant miss events cause the OOO data structures to fill up. Detailed analysis is hence limited mainly to finding miss events. In contrast, the ROB timer simulation mode mainly proceeds cycle-by-cycle and individually models each instruction in the ROB. We decided to go with the ROB model because of its higher accuracy and more detailed view of instruction execution provided.

4.2.1 Accounting core accesses

The bulk of the MLP accounting was implemented in a function `countOutstandingMemop()` which is run every simulation interval in the main loop in Sniper that simulates the ROB. Each function call, we iterate over the entries in the ROB and identify loads which are marked as currently executing in that simulation interval. In fact, at this stage we are able to retrieve which level of the cache hierarchy each of these loads hit at. This is because Sniper simulates the memory subsystem separately to the core processor and in fact simulates each memory access completely from start-to-finish at time of issue, adding contention penalty factors after the fact to account for parallelism (even though the simulator itself simulates each load one-at-a-time). We can hence count how many loads are at each part of the memory hierarchy here.

The simulation tracks a number of integer "MLP histogram" counters which are organised into an array of counters for each level of the memory hierarchy (including DRAM), each entry of which we will denote as $MLPHist[L][N]$. The accounting then proceeds in two phases. In the first phase, for each memory level, we count how many memory accesses there are which reach each level L by iterating over the ROB. Then, in the second phase, for each L we add the current simulation interval's time to $MLPHist[L][NumAccesses[L]]$. Thereby, $MLPHist[L][N]$ comes to mean the time spent having N parallel accesses at level L. We also count the total amount of time spent having an access to any level of the memory hierarchy, which is the denominator we will end up averaging over.

In post processing, we then compute the core components of total cache-level parallelism $TCLP$ and DRAM-level parallelism as below. The sum at the top is of the number of time spent having N accesses times N, which is in fact the same as the sum of the number of accesses at any given time over time (i.e. $\sum_t MLP(t)$), verifying that this

formula matches our normal formula for average MLP as in (1).

$$MLP_C = \frac{\sum_N MLPHist[DRAM][N] * N}{T(Hier)}$$
$$TCLP_C(Ln) = \frac{\sum_N MLPHist[Ln][N] * N}{T(Hier)}$$

We also extract $MCLP$ and $HCLP$ via the following formulas. These make use of the fact that the consistent denominator allows us to apply relationships between the numbers of hits, misses and total accesses. Then, the number of (core) misses at a level is equal to the total number of accesses that reached the next level (and had to miss this level), while hits = total - misses. $L+1$ denotes the next level, which in the case of last level cache is DRAM.

$$MCLP_C(L) = TCLP_C(L+1)$$
$$HCLP_C(L) = TCLP(L) - MCLP_C(L)$$

4.2.2 Accounting prefetches

Prefetches in Sniper are modelled inside the memory subsystem. Their execution is not handled differently from a normal memory access, which are both passed from cache level to cache level through the various functions in the cache controller part of the simulation, except they are injected at a particular level by `doPrefetch`. During simulation of the prefetch the current processor time is advanced, and while it is reset before continuing simulation, the amount by which it advanced corresponds to the interval of time over which the prefetch was executing. We add a mechanism via a global buffer for the cache controller to broadcast information about each prefetch, including which cache level it originated and hit at as well as what time the prefetch began and ended. In `countOutstandingMemop()`.

We then simply count prefetches if their execution time overlaps over the simulation interval currently being simulated in the ROB timer. We keep prefetch histogram counts similar to the MLP histogram counts for this purpose, and proceed in the same two-phase counting mechanism except we iterate through prefetches in the buffer instead of loads in the ROB. In post-processing we have the same formula for MLP_P and $TCLP_P$ except with prefetch histograms. For $MCLP_P$, we in fact keep the same formula and count accesses which originate at level L (i.e. from prefetchers at that level) as misses since they must be tracked in the MSHRs, even though they never actually check level L and so technically don't miss. $HCLP_P$ can be kept the same as well.

For identifying useful and useless prefetches, a prefetch is deemed useful if at some point later in execution there is a core access which requests the data retrieved by the prefetch and is able to be served by the cache using the prefetched data. A useless prefetch on the other hand occurs when the prefetched data is evicted before being used or never used. The publicly available version of Sniper already has a mechanism for counting useful

prefetches by setting a bit within the cache line retrieved by a prefetch, and checking this bit when processing an access to the cache. Meanwhile the bit is cleared upon eviction. We cannot in fact keep seperate histograms for useful and useless prefetches because one may not know if a prefetch is useful until after the ROB simulation time has passed over the prefetch. However, the easy fix is to make use of the existing counts for useful prefetches and multiply MLP_P by the ratio of useful to total prefetches, which is what we do.

4.2.3 Speculative MLP

Given that the core access MLP accounting already involves iterating over the ROB to check loads which are currently executing, it is relatively easy to extend this to check loads which are *not* currently executing. Specifically, we keep additional histograms for tracking time spent having N loads which are dependency bound, which is indicated in Sniper by the load having its "ready" boolean unset, and for having N loads which are structural (hazard) bound, which in Sniper is indicated by the load having its "ready" boolean set but its "issued" boolean unset (i.e. it is not being issued for some reason which is seperate to its operands being ready).

4.3 MLP visualisation design

We finally discuss how all of these auxillary MLP metrics are combined into the final visualisation.

One aspect which we have not yet addressed is Motivation 1: indicating the extent to which MLP actually contributes to performance. The way we achieve this is by interposing information about MLP in our visualisation with the CPI stack. As discussed in the background, the CPI stack shows the total runtime of the program split into estimates of how much of that runtime was spent by the core smoothly executing instructions (dispatching new ones every cycle), and how much is spent stalled due to various bottlenecks. These bottlenecks include time spent waiting for accesses to various memory levels. This hence provides a first-order indication of MLP's connection to performance, as we can see the actual performance loss as a result of waiting for memory accesses, and as a result of insufficient MLP and thus insufficient overlapping of latencies while waiting.

The visualisation is split into three graphs: the hit stack, miss stack and total stack. Each of these graphs has boxes corresponding to the following components:

- DRAM. In all three graphs, total DRAM MLP is shown.
- Cache components. In each graph, a different type of CLP is shown, i.e. total stack shows TCLP, miss stack shows MCLP and hit stack shows HCLP.
- Speculative MLP.

- Compute. This block does not have much meaning in the MLP dimension, but in the CPI dimension it represents the runtime that is attributable to non-memory components.

In each graph, the horizontal dimension provides information about MLP. In the horizontal dimension, the DRAM and cache component boxes have width corresponding to their average parallelism (MLP/XCLP). This parallelism is also split horizontally between the core, useful prefetch and useless prefetch components using different brightnesses of colors, with the total width indicating the total parallelism (core + prefetch) at that level. In the miss stack the parallelism shown horizontally is MCLP which indicates MSHR occupancy while in the hit stack it is HCLP which indicates overlapping of accesses to that level.

Meanwhile, the vertical dimension provides information about CPI. The DRAM and cache component boxes have heights equal to the number of cycles where there was at least one access being handled there, indicating the total time they each spent working. Then the vertical position of each of these boxes corresponds to its CPI component, with each box overlapping with the next and the positions being such that the distance from the bottom of each box to the bottom of the next box, i.e. the exposed vertical distance, is equal to the CPI at that level. For instance, L1 box begins at vertical position 0 while L2 box begins at vertical position $CPI[L1]$ (making the exposed height of the L1 box $CPI[L1]-0 = CPI[L1]$), and then L3 box begins at vertical position $CPI[L1]+CPI[L2]$ (so the exposed height of the L2 box is $CPI[L1]+CPI[L2]-CPI[L1] = CPI[L2]$) and so on. The DRAM box then is overlapped by the compute box, whose height represents the remaining non-memory CPI. The hidden part of the height of each box then represents the time where that memory component was active, but which was not seen by the processor due to it being overlapped with computation. This is explained visually in Figure 4.2.

The speculative MLP components have their exposed height given by RS-full, which is a compute component of the CPI stack corresponding to the issue queue becoming full, which unresolved loads in the issue queue contribute to. Its height is considered as part of the compute CPI component (the actual purple compute box has the rest of the compute CPI), while the width shows MLP.

Figure 4.1 shows an example MLP stack graph with annotations summarising the explanation given above in text.

The graph itself was implemented using Matplotlib as part of the same Python script used to do post-processing on the MLP counters.

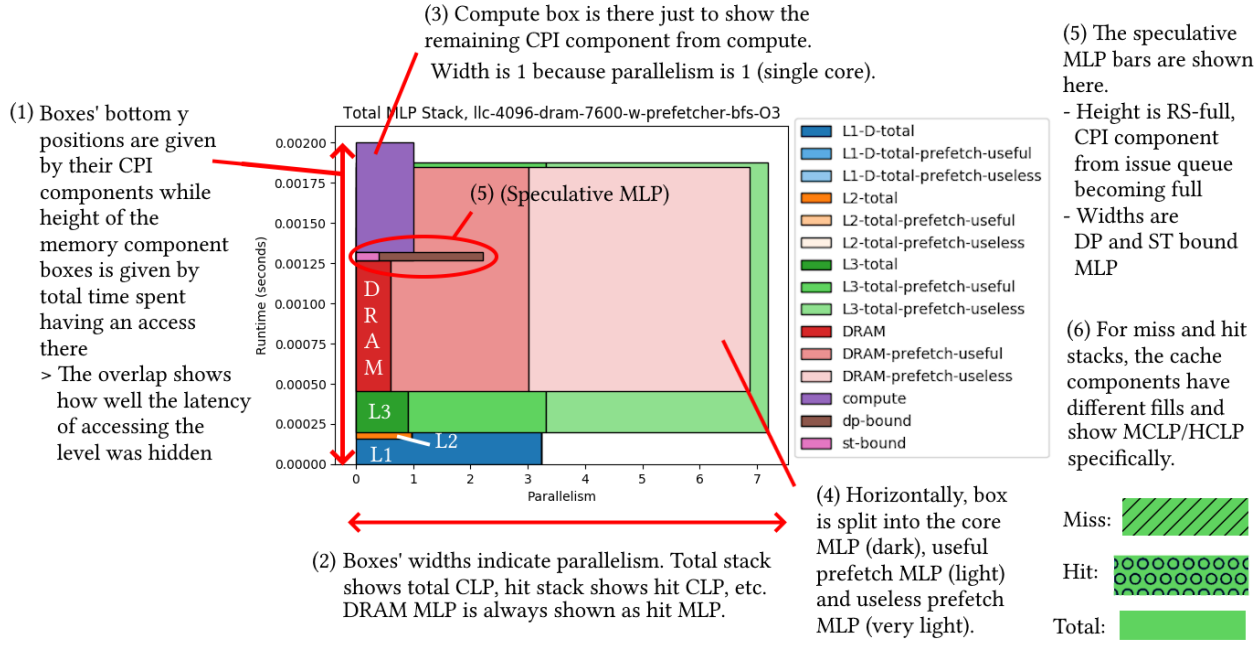


Figure 4.1: Visual explanation for an example MLP stack graph. See the text for more details.

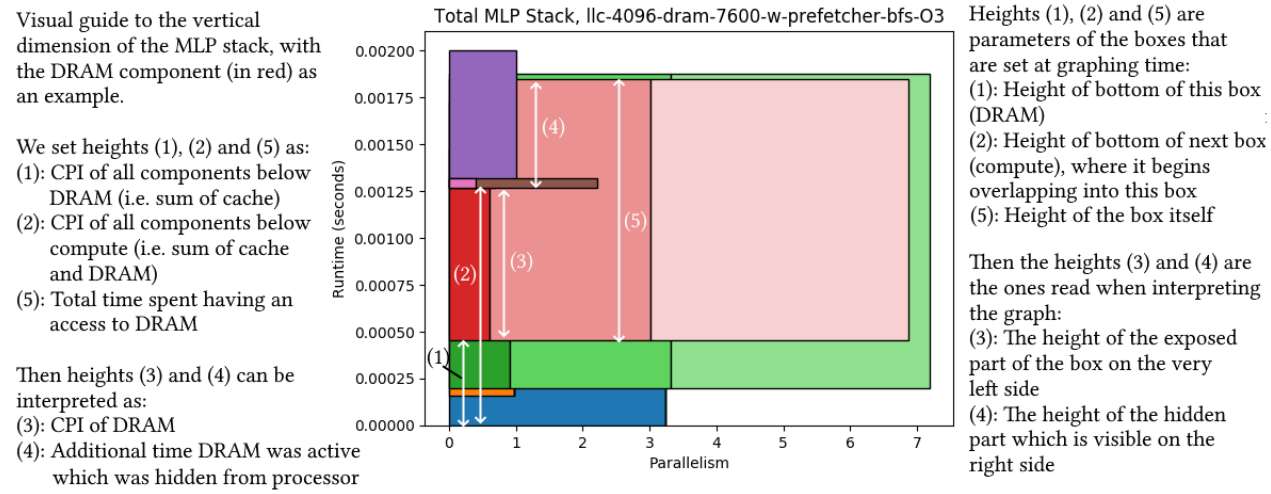


Figure 4.2: Visual explanation of the vertical axis of the stack graph specifically. See text for more details.

Chapter 5

Experimental Methodology

5.1 Default hardware

The default hardware we simulated was the Gainestown processor used in the 5500 series Intel Xeon cores, which is based on the Nehalem microarchitecture. Sniper came provided with a configuration file including the specifications of this processor, collected with a combination of direct benchmarking and from online manuals/specifications, which we used. A subset of the parameters can be found in Intel (2009). The relevant details of the processor configuration are described in Table 5.1. Since our methodology is limited to modeling MLP for single-core systems, we used a single core version of Gainestown.

We did make one small addition to the existing Nehalem and Gainestown configuration for our default, which was to limit the MSHR capacity. This is done through the "outstanding misses" parameter in Sniper, which is usually set to 0 (no limit). We set outstanding misses to 16 for L1. We left the L2 MSHR size uncapped based on the idea that it is larger than L1 and unlikely to be reached. In retrospect these choices were somewhat arbitrary and should have been examined more closely, but due to time constraints left largely fixed.

In some experiments we enabled hardware prefetchers. Our with-prefetcher configuration consisted of a simple linear stride prefetcher at L2.

5.2 Benchmarks

Our experimental case studies focused on two particular benchmarks. The first benchmark used is a simple sparse matrix-vector multiplication (SPMV) program. This benchmark included the implementation of non-trivial software prefetching. This should have injected additional MLP, allowing us to test the stack by seeing if we could observe this additional MLP and the associated performance benefit. The second benchmark is the

Table 5.1: Default processor parameters used in simulations

Processor	
Processor	Intel Xeon X5550 Gainestown
Number of cores	1
Core frequency	2.66 GHz
Issue width	4-wide
ROB size	128 entries
Branch predictor	Hybrid local/global predictor
Max. outstanding	48 loads, 32 stores, 16 L1-D misses
Cache Hierarchy	
L1-I	32 KB, 4 way, 4 cycle access time
L1-D	32 KB, 8 way, 4 cycle access time
L2 cache	256 KB, 8 way, 8 cycle access time
L3 cache	8 MB, 16 way, 30 cycle access time
Cache block size	64 bytes (for all levels)
Prefetcher	
L2	Simple linear stride
Enabled?	Varies by experiment
DRAM	
Bandwidth	7.6 GB/s
Latency	45 ns
DIMMs	4 DIMMs
Chips	8 chips/DIMM

Graph Application Processing Benchmark Suite, or GAPBS, described in Beamer et al. (2015). These are small kernels which implement common graph applications, such as breadth-first search or finding the connected components of a graph, designed to be optimised and provide a baseline for graph researchers. We use these to test our stack on larger real life applications and observe what they can tell us about the performance of a program. Our exploration of GAPBS also involved changing some hardware and software parameters to evaluate the impact on the stack.

5.2.1 SPMV

Our SPMV benchmark is not publicly available as we used a custom kernel provided by one of the project's supervisors Wim Heirman. Sparse matrix-vector multiplication is a very common application that is readily found however. Nevertheless, we made a minor edit to add software prefetching, which we describe below.

The benchmark completes a matrix-vector multiplication of a matrix M with a vector x, but where the matrix M is sparse and represented in the compressed sparse row (CSR) format. The CSR format greatly reduces data usage by storing information about the nonzero values only. The format consists of three one-dimensional arrays V, CI and RI, where V and CI have length equal to the number of nonzero values and V stores the nonzero values in row-column order while $CI[i]$ is which column in the matrix entry $V[i]$

```
for(int row = 0; row < nrows; ++row) {
    row_result = 0;
    for (int i = RI[row]; i < RI[row + 1]; i++) {
        row_result += V[i] * x[CI[i]]                # (*)
    }
}
y[row] = row_result;
```

Figure 5.1: C code for Sparse Matrix-Vector multiplication

```
for(int row = 0; row < nrows; ++row) {
    row_result = 0;
    for (int i = RI[row]; i < RI[row + 1]; i++) {
        __builtin_prefetch( &x[CI[i + PREFETCH_DISTANCE]] );
        row_result += V[i] * x[CI[i]]
    }
}
y[row] = row_result;
```

Figure 5.2: Our adjusted SPMV kernel including non-trivial software prefetching

is at, while RI has length equal to the number of rows and entry $RI[i]$ gives the entry in V where row i begins (so $RI[i+1]$ gives where the next row begins and this row ends). For the multiplication $y = Mx$ we can compute the ith element of y as the dot product of row i of M with x. If M is in CSR format and x is an array, we can in fact do this computation without decompressing M like shown in Figure 5.1.

The key line of computation is marked by the comment (*). For each row, we must multiply each entry in column k of M with the kth entry of x and sum the results. The key insight is that we only need to multiply the nonzero entries in the row of M. VI[i] is the entry in M at column CI[i], and so needs to be multiplied with the CI[i]th entry of x, which is what line (*) expresses.

All of this is generally standard implementation for this kernel. Where our benchmark differs is the insertion of software prefetches. The benchmark iterates through V, RI and CI linearly and in order, which allows linear hardware prefetching to easily prefetch accurately for these arrays. However, the array access to x given by $x[CI[i]]$ is nonlinear because the index $CI[i]$, while increasing monotonically since column indices are generally sorted for each row, may increase in erratic jumps based on the specific indices, which the hardware prefetcher will be unable to predict. However in software we do know that the next value of x accessed at each point is $x[CI[i+1]]$, and so we can insert a software prefetch like shown in Figure 5.2.

We use the following default parameters with this kernel:

- Matrix and vector entries were doubles.
- Length of vector x: 2^{25} entries. Each entry is a double so this leads to a vector size of $2^{28} = 256$ MB, which indeed does not fit in our LLC. The vector size was chosen by checking the runtime of the program on native hardware and estimating simulation time accordingly, and choosing the largest size that still has a tractable simulation time. This also dictates length of RI.
- Number of nonzero entries in matrix M, which dictates length of V and CI: 2^{25}.
- Run two iterations with only the second one in Sniper ROI, with the first iteration to warm up the simulator (but disabling cache warming).
- Pseudo-randomize x and M, but use a set seed for reproducibility.

We compiled the SPMV benchmark with GNU GCC 11, using O3 optimisation level. This is mentioned because the `__builtin_prefetch()` function's implementation is compiler specific.

5.2.2 GAPBS

We used the v1.3 (ISCA 2021) release of GAPBS as available on the Github repo at Beamer et al.. For each benchmark we used a generated uniform-random graph with 2^{16} vertices and the default node degree setting. We used Sniper region of interest (ROI) markers in order to exclude the graph generation step from being simulated in detail as well as from the results. The GAP benchmarks were compiled with the `SERIAL=1` to account for the processor configuration having a single core, since otherwise the benchmark is compiled to use as many threads as there are cores detected on the machine, and we were compiling natively on a server with 112 cores which would not have been appropriate.

We focus on the six main kernels included in GAPBS, which are as follows (as documented in Beamer et al. (2015)): Breadth-First Search (BFS), Single-Source Shortest Paths (SSSP), PageRank (PR), Connected Components (CC), Betweenness Centrality (BC) and Triangle Counting (TC). We use the main implementations and ignore secondary implementations of alternative algorithms that are provided in the suite.

We compiled GAPBS with GNU GCC 11, using the standard production compiler settings used in the Makefile (other than SERIAL=1).

Chapter 6

Results

6.1 SPMV

We first discuss our sparse-matrix vector (SPMV) benchmark case study for exploring the software-side optimisation of software prefetching.

Recalling our discussion of the custom SPMV benchmark with software prefetching from Chapter 5, we had a parameter for distance to prefetch N, and in each iteration i, at the same time as doing the nested array access $x[CI[i]]$ we would do a nontrivial software prefetch for $x[CI[i + N]]$. By setting N higher we prefetch a higher amount of loop iterations ahead. Our main experiment for this case study consists of comparing the program performance and MLP with no software prefetching, with software prefetching with various prefetch distances Ns. In particular, we increase the prefetch distance exponentially (1, 2, 4, 8, etc.) up to 128 in order to sweep the parameter space efficiently. We use the default processor configuration with L2 hardware prefetcher enabled.

Our hypothesis is that the additional software prefetches should inject additional MLP which should be visible in the graph. As prefetch distance is increased, the actual number of prefetches in the loop does not change (it is still one prefetch per iteration), but the prefetches' timing relative to the iteration which uses that data does change. If the prefetch is too late then the data will not arrive in time to be useful, but if the prefetch is too early then the data may be evicted from the cache by the time it is to be used and also become useless. We thus expected to see total runtime follow a performance curve, with prefetch distance where there is an optimal distance somewhere between too soon and too late. As for MLP, since the number of accesses doesn't change, we expect it to stay the same with prefetch distance.

The result MLP graphs are shown in Figure 6.1. We show just the total MLP graphs in this case because L1 MLP is obscured entirely by DRAM in the miss and hit graphs and is most easily visible in this graph. L1 miss MLP was almost equal to DRAM MLP

in all cases so one can interpret the "extra" horizontal component of L1 which is visible beyond DRAM as the hit MLP in this case. We also show the raw numerical results for runtime, L1 and DRAM hit MLP, and L1 hit rate in Table 6.1.

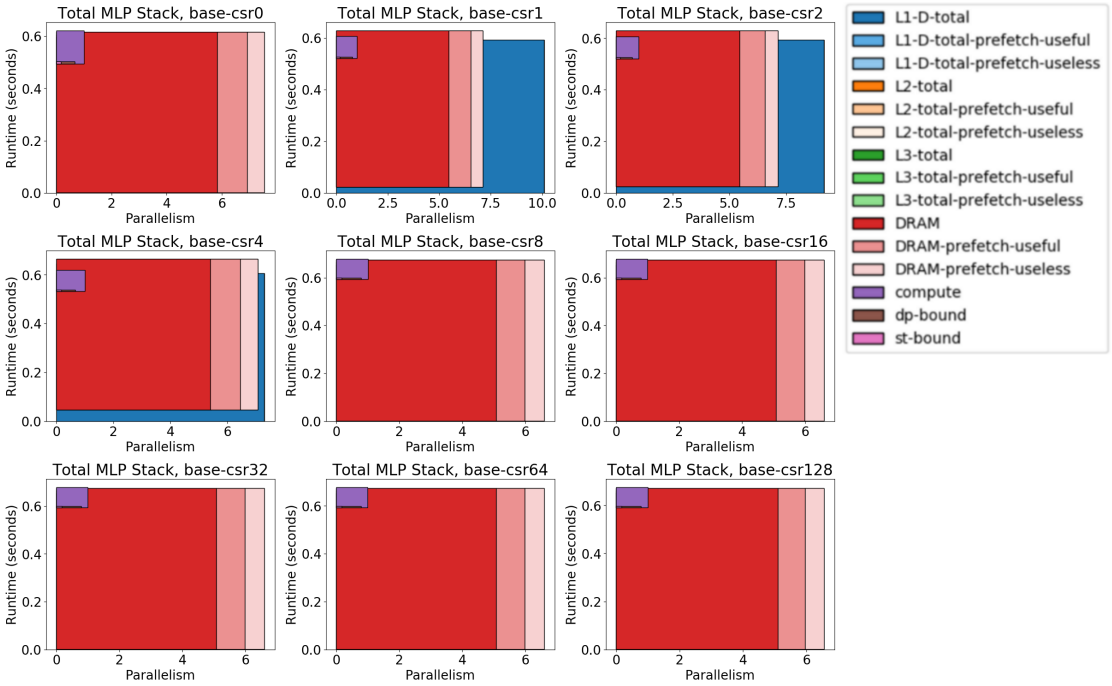


Figure 6.1: Total MLP stacks for SPMV benchmarks. The graph titled "base-csrN" has software prefetching with stride N, with the graph "base-csr0" having no software prefetching. Thus the top left graph shows no software prefetching, while each subsequent graph from left to right and top to bottom shows increasingly early software prefetches.

Table 6.1: Raw numbers from SPMV experiments (important metrics)

Prefetch distance	Runtime (s)	L1 hit CLP	L1 miss rate	DRAM (hit) MLP
0	0.622	0.765	22.92%	5.858
1	0.607	4.161	17.19%	5.458
2	0.606	3.701	17.19%	5.445
4	0.621	1.873	17.19%	5.402
8	0.678	0.881	17.19%	5.104
16	0.678	0.874	17.19%	5.102
32	0.678	0.874	17.19%	5.102
64	0.678	0.883	17.22%	5.102
128	0.678	0.960	18.60%	5.129

We indeed observe the expected performance curve, with runtime decreasing from 0.62 s to 0.60 s after enabling prefetches, staying constant as prefetch distance is increased, and then increasing again to 0.67 once prefetch distance exceeds 8 and the prefetches are no longer timely. Interestingly, there is no "too soon" regime observed. On the vertical dimension of the MLP graphs, we can observe that the program is very highly DRAM bound, with the CPI stack developing a small L1 component after enabling prefetches but still remaining mostly DRAM bound. A small portion of the runtime is also compute. Note that in the MLP stack, the total runtime is actually given by the top of the *compute* component.

Returning to the analysis of SPMV, we would like to connect the performance behavior observed with MLP. We first focus on analysing the effect of turning on software prefetches (prefetch distance 0 vs prefetch distance 1). L1 HCLP dramatically increases from 0.765 to 4.161 when we turn on prefetches. This is complemented by the L1 miss rate decreasing from 22.92% to 17.19%. At the same time, DRAM hit MLP decreases from 5.858 to 5.458, and the miss CLP at all levels similarly decreases. This partially aligns with our hypothesis as MLP does increase, but we observe it increase specifically at the L1 level, and in fact DRAM MLP decreases. Since the prefetches will fetch data into cache which the core accesses will hit on, we can safely say that the core accesses contribute the extra L1 HCLP observed and do not contribute DRAM MLP anymore because they do not go to DRAM. Our hypothesis is hence half correct; parallelism does increase, but it increases at L1 HCLP because of the core accesses hitting there rather than due to the prefetches themselves. The prefetches do still contribute DRAM MLP, but in fact *less* MLP than we had from the core accesses. Looking at additional metrics we find that the actual number of accesses to DRAM only increases by 100 compared to the total number of 50 million accesses, which indicates that the change in MLP is not due to there being less DRAM accesses overall. Rather, we may suppose that the clustering/timing of these accesses is different and leads to less MLP. Nevertheless, the lower DRAM MLP does not impact performance, as while the prefetches overlap less with other loads, they overlap more with compute. In some respects our graph is effective here as DRAM MLP measurement alone would suggest lower performance, but considering the cache effect allows us to observe that the core accesses have been converted into L1 HCLP which we can see on the graph, which does improve performance.

As prefetch distance is further increased, L1 hit CLP decreases drastically, returning to similar levels before turning on prefetches at around distance 8. This is also around where runtime spikes up to 0.678s and becomes worse than without prefetches. We initially suppose this was due to prefetches becoming too early, causing the prefetched data to be evicted before use. However, L1 miss rate in fact does not change at prefetch distance 8, which is rather odd and contrastingly suggests the prefetches remain timely. If the data was being evicted then we would expect the HCLP at L1 to move to L2 and L3 as the data should still be found at the larger caches, but in fact L2 and L3 HCLP remain low at all prefetch distances, and while the number of misses at L2 does drop from 39.6 million to 39.0 million after turning on prefetches, it increases to 39.7 million

at distance 8 and does not behave differently from L1. The question then remains: if the data is not being evicted and miss rates aren't changing, why does the performance degrade at prefetch distance 8?

In this case, we observe that DRAM hit MLP in fact also decreases at prefetch distance 8. Hence, we may have an alternative explanation for the increase in runtime in terms of MLP; at this distance, the clustering of accesses to DRAM changes again, and this time it does in fact affect the performance negatively. We also need to explain the L1 HCLP decreasing when the L1 miss rate does *not* decrease. However, considering the frequency of hitting accesses to L1, it theoretically should not change since the timing between the hitting accesses should remain one-per-iteration of the loop. Further investigation is needed to understand this behavior, likely involving analysing the clustering of accesses more closely.

6.2 GAPBS

We now discuss our results from the GAP benchmark study. This study was more free-form as we did not have a particular optimisation in mind to study, and we instead proceeded with the idea that we would observe the MLP graphs and consider the scope for MLP improvements which they indicated. We then later used this data to action some hardware optimisations, which relied on using the speculative MLP results.

6.2.1 Initial exploration

Our exploration of this benchmark suite first began with a parameter search in order to identify the base behavior of the benchmarks, and verify the MLP metrics behaved in a reasonable manner. We evaluated all six benchmarks while varying the following parameters:

- DRAM Bandwidth: We scaled down DRAM bandwidth to 1/2×, 1/4× and 1/8× the base level supported by the processor of 7.6 GB/s. For the experiments where this was varied, LLC size was fixed.
- LLC Size: We scaled down the LLC size to 1/2×, 1/4× and 1/8× the base amount of 8 MB as well. For the experiments where this was varied, DRAM BW was fixed.
- Optimisation level: We varied the optimisation level used to compile the benchmarks. We will show results for O0 and O3 mainly as O1 and O2 were found to be similar to O3.

We ran all of these configurations with and without hardware prefetchers as well.

Note that there are far too many MLP stack graphs for us to be able to display them all in this section, so we display a representative set of results. The discussion may cover additional graphs not shown. The artifact repository contains all result graphs (in addition to raw results) for completeness.

Comparing benchmarks

We show the hit, miss and total MLP stack graphs for each benchmark without prefetches in Figure 6.2. By reading the vertical component as a CPI stack, we observe that the benchmarks BC, BFS, SSSP and CC are all memory bound while PR and TC are mainly compute bound. Observing the compute bound benchmarks, we can see the L2 and L3 bars stretch up high beyond the bottom of the DRAM bar in PR, which suggests that there are many memory accesses in PR but they generally hit in L2 and L3 and their access time is hidden by compute. Meanwhile, TC is compute bound but does not show much memory activity at all except at L1. There is no point optimising memory performance and MLP/CLP if memory/cache access cost is already sufficiently covered by compute, so PR and TC can be excluded from consideration for now. Meanwhile, the memory bound benchmarks are split between L3 bound and DRAM bound, with SSSP being the most DRAM bound at these settings.

On the MLP side, we don't see very much MLP in even the memory bound benchmarks. The DRAM MLP is close to 1 in all of them. In the miss stacks, we do observe MCLP at L1 and L2 as higher than at DRAM due to contribution of accesses that hit at L3; however, it is nowhere close to the MSHR capacity of 16 in both cases. This indicates large scope to improve the MLP of the programs without the MSHRs posing a bottleneck.

Scaling DRAM BW

Figure 6.3 depicts the effect of scaling the DRAM bandwidth on a representative benchmark (BC). On the CPI side/vertical side, total runtime increases while the DRAM component becomes progressively larger, which is as expected since DRAM access latencies worsen and the program is more likely to become DRAM bound. On the MLP side, we observe that L3 hit CLP decreases while DRAM MLP increases as we decrease DRAM bandwidth. As for miss CLP, it decreases at L1 and L2 while increasing at L3. We expect DRAM MLP to increase since the greater access latencies will cause accesses to overlap more, but L3 hit CLP decreasing is unexpected as accesses which hit at previous levels should be unchanged. In addition, we generally expect miss CLP to increase because the misses which reach DRAM take longer, which we observe for L3, but L1 and L2 miss CLP in fact decrease.

The anomalous behaviors in CLP are rather confusing, but we eventually found that the reason was changes in the denominator which we were averaging MLP over. Recall that our MLP accounting involves tracking a cumulative record of instantaneous CLP, which is then averaged over the time spent having at least one access to the memory hierarchy. This total time had gone from 0.016 s with DRAM B/W of 7600 MB/s to 0.025 s with DRAM B/W of 950 MB/s, as a result of increased DRAM latencies. In fact, the numerator in the equation, the cumulative CLP, did follow the trends we had expected; MCLP at L1 and L2 increased, and HCLP at L3 did not change. This is an important problem as it indicates that changes in the denominator we chose to average CLP/MLP over can have unintuitive effects on the results which obscure the actual

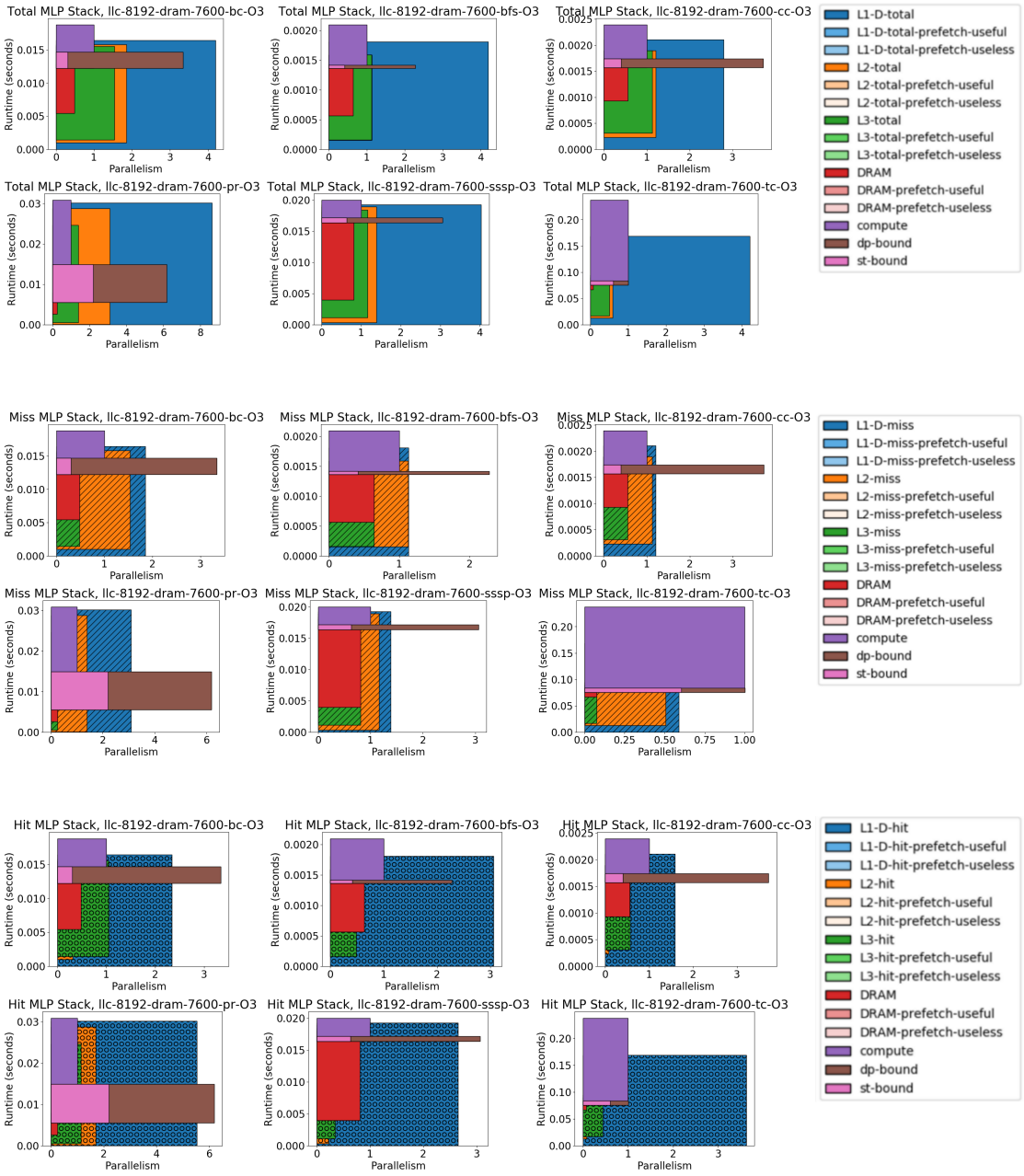


Figure 6.2: Total, miss and hit MLP stacks for GAPBS, comparing benchmarks. Top is total stacks, middle is miss and bottom is hit. The graph titles of "llc-8192-dram-7600-XX-O3" indicate which benchmark is shown in each graph, which is one of BC, BFS, CC, PR, SSSP and TC (see details of benchmarks in Chapter 5). The title also shows the LLC size, DRAM B/W and optimisation level, which are constant across experiments here.

parallelism effects we are trying to study. We will discuss this further in the limitations section of the conclusion.

Across the other memory bound benchmarks, we observed the same general effects.

Scaling LLC size

Figure 6.4 depicts the effect of scaling the LLC size for the BC benchmark. On the vertical part of the stack we observe that the DRAM CPI component decreases while the L3 CPI component increases, with the runtime increasing dramatically. This is as expected since the L3 hit rate decreases and more accesses go to DRAM, incurring the high DRAM latency. We correspondingly see L3 HCLP decrease while DRAM MLP increases, as expected. L1 HCLP does not change which is as expected since accesses that hit at L1 are not changed. For MCLP, L1 and L2 slightly increase while L3 MCLP increases significantly. We in fact expect the increase at L1-L3 to be similar across levels since the increase of DRAM MLP should be equally seen at all prior levels.

In fact, upon further investigation, this benchmark does suffer from the same shifting denominator issue observed for the DRAM B/W results. Yet, in this case, the results do not seem as greatly affected by it, or at least the trends observed match expectation. It is possible that in this case, while the time spent having at least one access to the memory hierarchy does increase, parallelism also changes proportionally to this time, and thus the behavior is generally more reasonable here.

Results are also generally similar for other benchmarks.

Changing optimisation level

Figure 6.5 compares the MLP stack for the BC benchmark compiled with O0 with that compiled with O3. When compiling with O0 the program puts all function local variables on the program stack, which leads to many more memory accesses, but which are predictable and have high locality. We indeed observe on the MLP stack that when using O0, the L1 component is dominant both in HCLP and CPI, which is consistent with our expectations. The MCLP components and DRAM MLP are very small in this case. This indicates that O3 makes better utilisation of MLP than O0, which we can expect as well since the O0 program is likely to have many long dependency chains as a result of constant stack accesses, which are serialised in MLP.

We also note that we observe that in the O0 configurations, the speculative MLP component for ST-bound is relatively large. This will be touched upon later.

The other O0 benchmarks behave similarly.

Effect of hardware prefetchers

The total, hit and miss MLP stacks of each benchmark at O3 level with standard DRAM and LLC provisions are shown in Figure 6.6. The compute bound benchmarks PR and

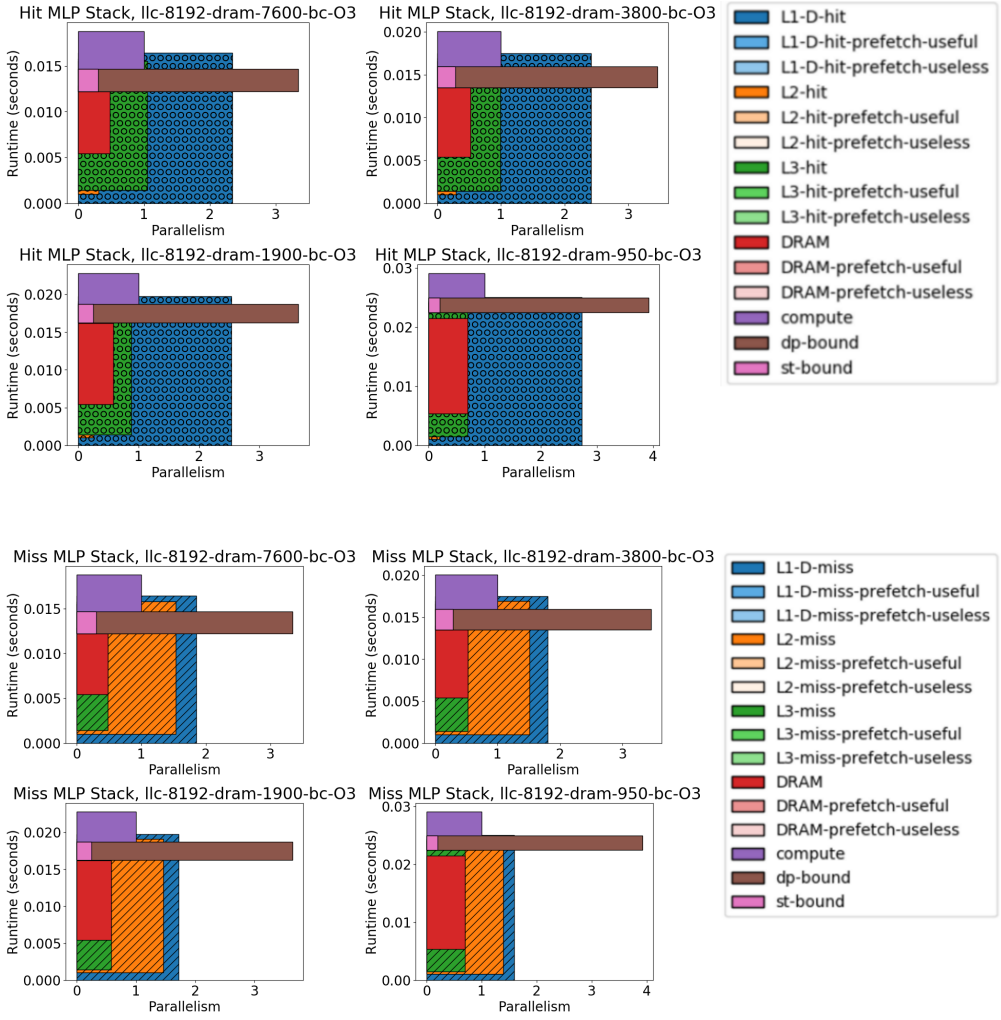


Figure 6.3: Hit and miss MLP stack after scaling DRAM bandwidth for BC benchmark. The graph titles of "llc-8192-dram-XXXX-bc-O3" indicate the DRAM bandwidth used in each graph in MB/s, e.g. 7600 indicates bandwidth of 7.6 GB/s.

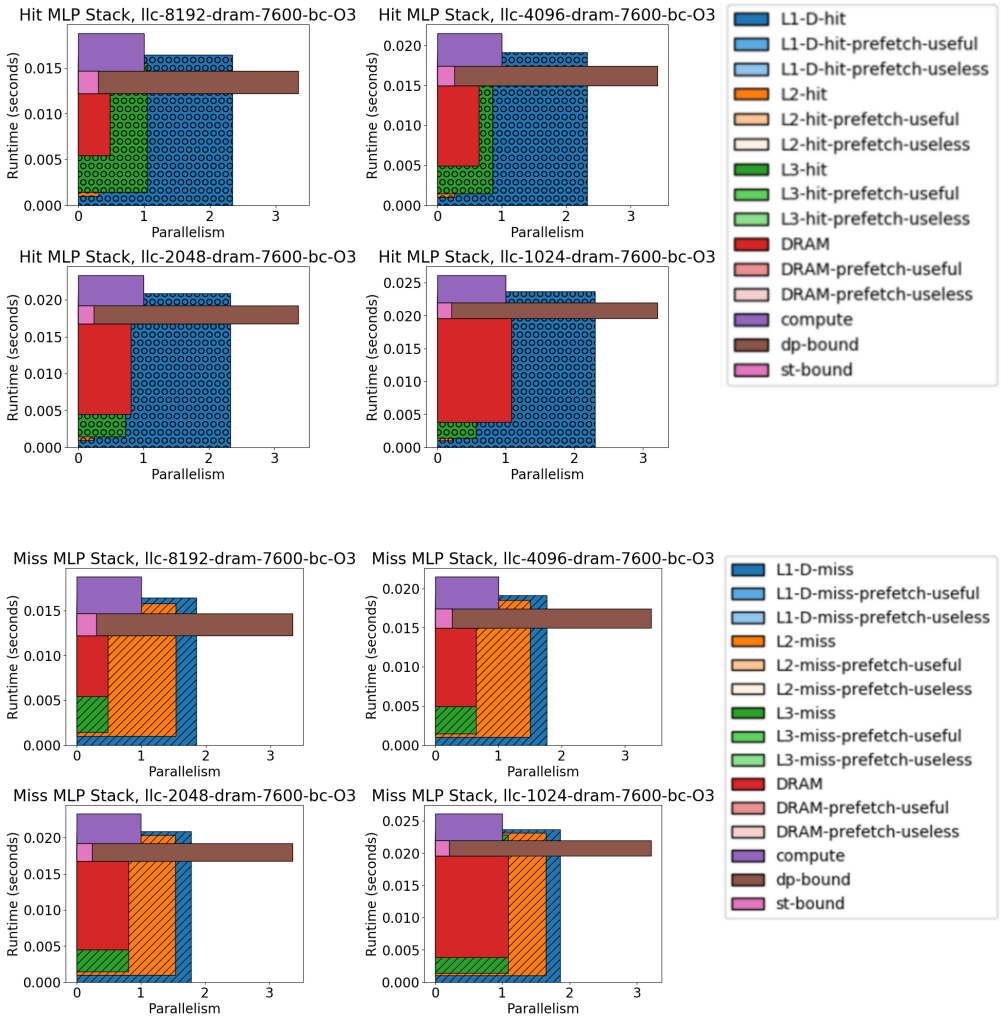


Figure 6.4: Hit and miss MLP stacks after scaling LLC size for BC benchmark. The graph titles of "llc-XXXX-dram-7600-bc-O3" indicate the LLC sizes in KB showed in each graph.

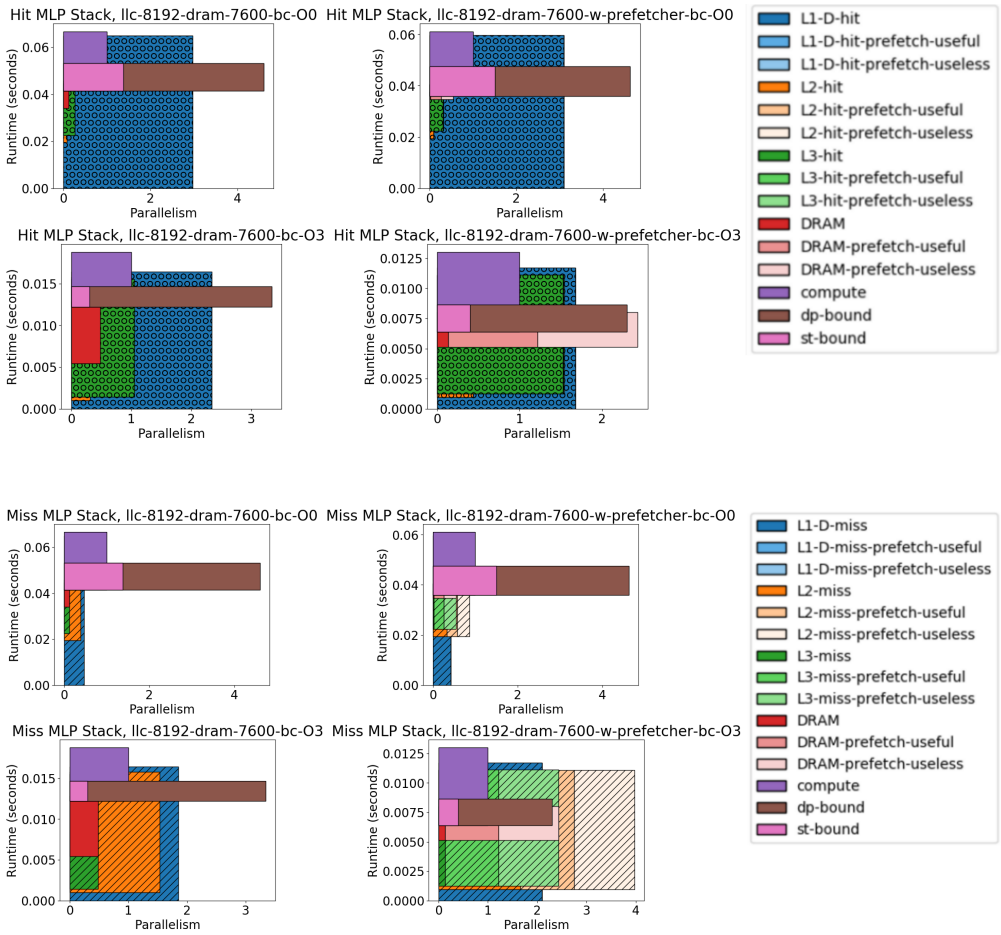


Figure 6.5: Hit and miss MLP stacks for BC benchmark with varying optimisation level, as indicated by the graph titles (O0 or O3). We also show the versions after enabling prefetchers here, though that will be discussed more in depth in the "effect of hardware prefetchers" section.

TC do not exhibit much additional MLP from prefetches, but we can see additional MLP in the memory bound benchmarks, with the useful component of prefetch MLP ranging from 1 to 2 additional units. The core component of DRAM MLP is in fact smaller in these configurations than without prefetches, which can be explained by core accesses hitting prefetched data in caches instead of going to DRAM and thus the core MLP being partly converted into useful prefetch MLP. We can also see a large amount of useless prefetches; for BFS for instance, over half of the prefetch MLP is useless. Theoretically, if too much of the injected MLP is useless then the benefit from core accesses hitting prefetched data in caches is outweighed by the increase in latency of accesses to DRAM as a result of contention overheads between concurrent accesses to DRAM, which is an effect that our stack actually does not display, which represents a gap in our work.

6.2.2 Using speculative MLP

Having done our initial parameter sweep and observed the basic properties of the stack, we proceeded to attempt using the stack in order to guide optimisation. We recall that we observed that L1 MSHR capacity had not been reached and there is scope for more MLP. For this purpose, we decided to do a case study on using the speculative MLP components, which should have indicated scope for additional MLP which we can exploit.

Most of the benchmarks except TC bound have a large DP-bound component of speculative MLP, indicating additional MLP from dependency bound loads in the ROB. This could potentially have been exploited, but since the dependencies of a program are software dependent and require reorganising the program to change, this would require some sophisticated work such as taking finer control over compiler optimisations for reorganising memory accesses. Exploring this is possible, but ended up outside of our timeframe. Instead we decided to target some low-hanging fruit in the form of the pink ST bound component, since these loads were blocked by structural hazards, i.e. insufficient processor resources to issue. Our hypothesis is that in the benchmarks with large ST bound component, by providing the processor additional resources, we may be able to allow these loads to issue and thus make use of additional MLP.

Our methodology for providing additional processor resources was varying the provisioned space in a couple of the OOO data structures through the simulator. In particular, we increased the sizes of the load queue, issue queue and store queue, with the idea that one of these may resolve the bottleneck stopping the ST-bound loads from being issued. Our other requirement was to identify a benchmark with large ST-bound component. PR with O3 has one, as shown in Figure 6.2. However, PR is compute bound and hence extra MLP would not help in that case. Upon closer inspection, we found that the O0 configurations with limited DRAM (950 MB/s) did have a decent ST-bound component while being compute bound. While O0 is a rather artificial example as programs are typically not compiled with O0 in real life, we just needed an example for demonstrative purposes, and so considered this fine. Thus we proceeded to do this experiment on the BC benchmark compiled with O0.

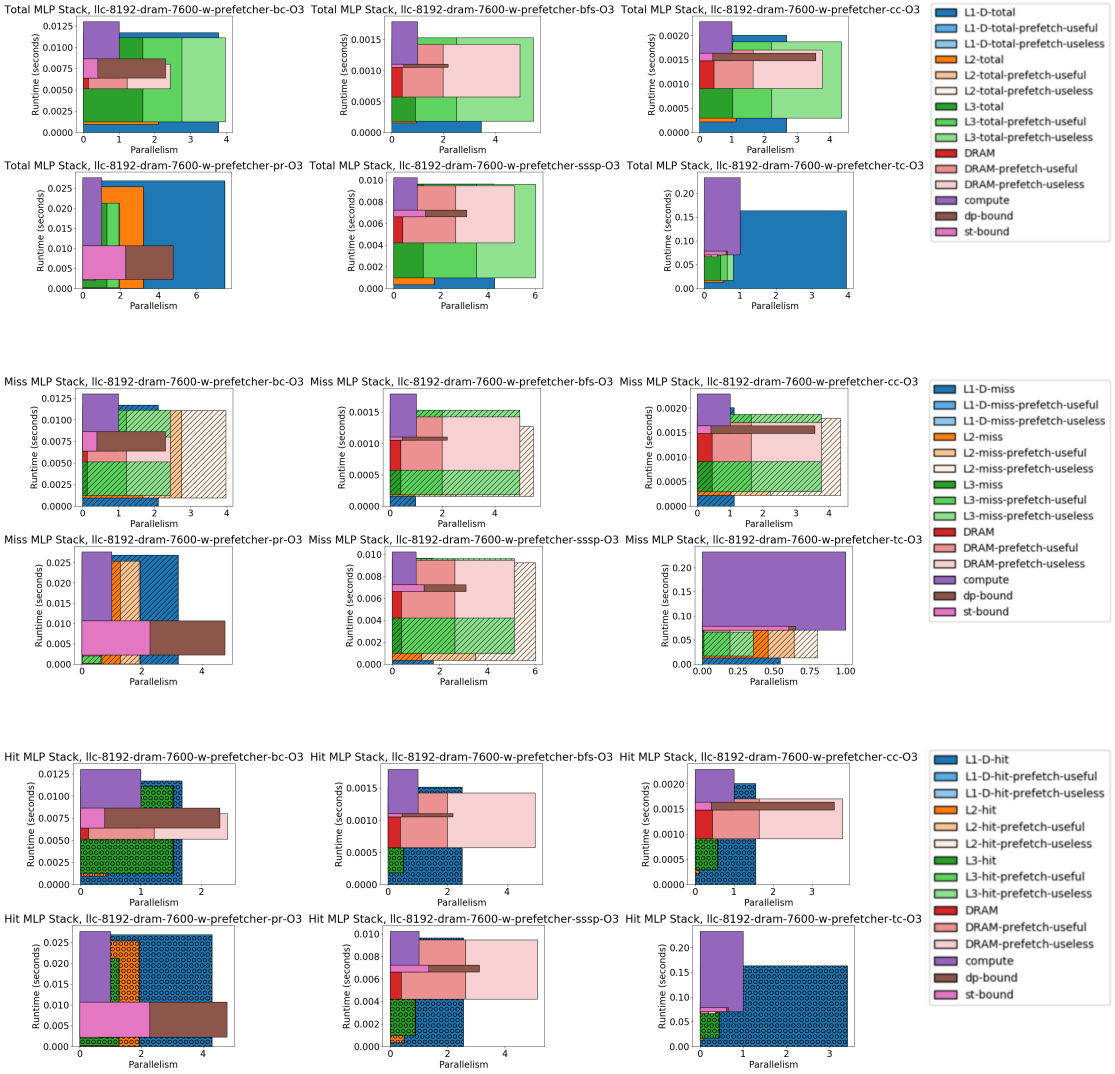


Figure 6.6: Total, miss and hit MLP stacks for GAPBS, comparing benchmarks. Top is total stacks, middle is miss and bottom is hit. The graph titles of "llc-8192-dram-7600-XX-O3" indicate which benchmark is shown in each graph. The LLC size, DRAM bandwidth and optimisation levels are constant across experiments here.

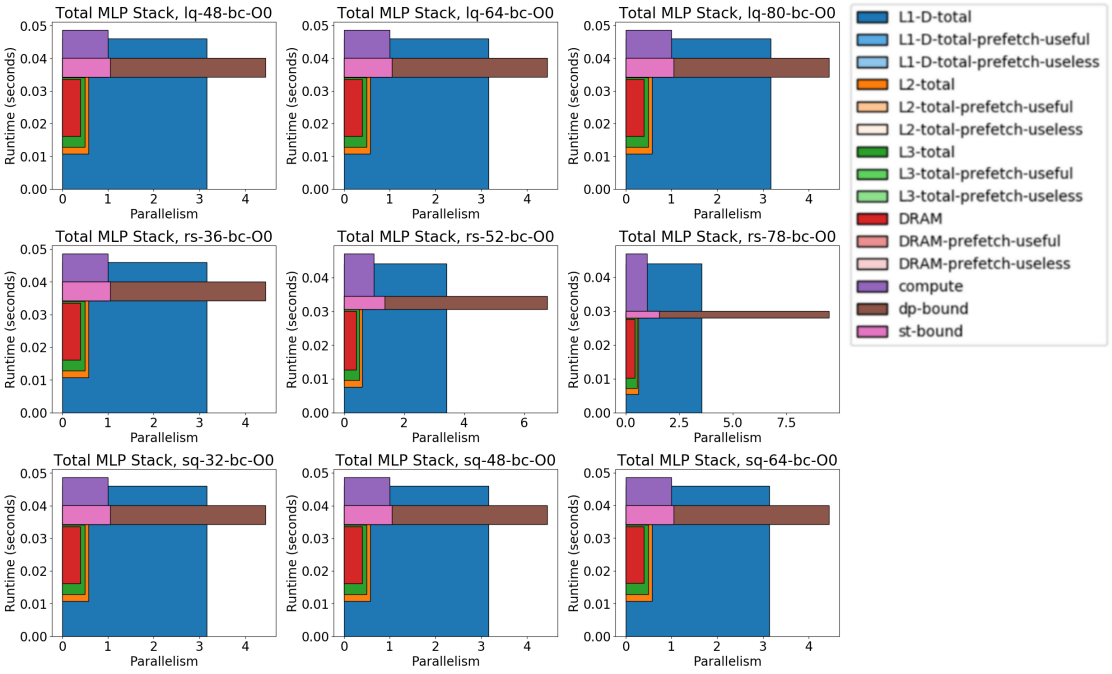


Figure 6.7: Results from varying processor resources on the O0 version of the BC benchmark in order to attempt to exploit the ST-bound speculative MLP. Top row is varying the load queue (LQ) size, increasing it from 48 to 64 to 80. Middle row is varying the issue queue/reservation station (RS) size, increasing it from 36 to 52 to 78. Finally, bottom row is varying the store queue (SQ) size, increasing it from 32 to 48 to 64.

The results are depicted in Figure 6.7. In fact, rather disappointingly, varying LQ and SQ size had no effect whatsoever. Meanwhile, increasing IQ size did have an effect and decreased the rs-full component of the CPI stack, visible in the MLP stack as the vertical height of the speculative MLP components decreasing. However, the ST bound component did not decrease in size, and MLP was also not affected. Thus, this experiment turned out to be a failure and we were not able to exploit the ST-bound component. Further experimentation or analysis in order to find the actual reason for the structural hazard that causes the ST-bound component may allow improving on these results.

Chapter 7

Threats to Validity

In this section, we address some inaccuracies in our model and methodological issues which may affect the validity of our results, and provide reasons why we believe they are not major problems. This does not involve addressing the limitations of the design of the MLP stack itself, which will be discussed separately in the conclusion.

7.1 Accounting error

When accounting MLP, since we construct the MLP histogram counters from iterating over the ROB, we in fact count each load as contributing CLP/MLP for all of the time which it spends in the ROB while executing. However, this strategy overestimates CLP/MLP because it includes all of the time the load spends in the memory hierarchy. For hit CLP at level L, the load should only contribute CLP for the time spent processing the hit at level L, but our accounting strategy also includes the time spent by the load going through previous levels in the hierarchy and missing at them. Meanwhile, for miss CLP at level L, the load should only contribute CLP for the time it spends being pending as a miss at level L, but our accounting strategy also includes the time it spent at previous levels as well as the time it spends being checked at the current level. The

We believe that the error from this is not substantial however because of two reasons. Firstly, latency of accesses at each cache level increases significantly going down the memory hierarchy. The error in each case is due to time spent at levels before level L, which is relatively small compared to the time spent at level L and beyond. For DRAM, it can be said that the total sum of cache latencies is minor compared to the DRAM latency. Secondly, the time spent processing a cache miss at level L is much less than the time spent processing a cache hit at level L because the former only requires checking cache tags against the memory address, while the latter involves actually retrieving the data from the cache. This is reflected in the fact that the cache access time parameter

in Sniper is split between tag access time and data access time; as an example, in the Gainestown configuration, tag access time at L1-D is 1 cycle while data access time is 4 cycles and is much greater. For both miss and hit CLP, since the error includes only time spent missing at levels and the actual time includes a hit and the associated data retrieval time, this is a further argument that the error is relatively small.

Nevertheless, the ideal solution would be to fix this accounting error. It would require more detailed analysis of the time spent by each load as it traverses the memory subsystem in Sniper, breaking the time down into time spent at each level. We leave this to future work.

7.2 Runtime of GAPBS kernels

It is noticeable that many of the GAPBS kernels have a very short total runtime on the order of hundredths of seconds. The reason why this occurred is that we set our input graph sizes to make the simulation time tractable for all of the benchmarks simultaneously. However, the benchmarks vary in their runtime on the same input graph size; in particular, the triangle counting (TC) benchmark is much more expensive than the others. Thus, this caused the shorter benchmarks to become very short. In many areas of experimental computer science, the interesting behavior only occurs at a large scale, and hence one may argue our conclusions regarding GAPBS are limited by the scale of our input sizes.

However, we are interested in the general behaviour of the benchmarks and what our MLP stack can tell us about it, which should still be possible to analyse even at small scale. Appropriately doing this requires accurately scaling the processor resources to match the experiment scale. While it can be argued that we in fact pay enough attention to this scaling, we were still able to extract some useful results from our case study.

Chapter 8

Concluding Remarks

We have presented an implementation of an MLP stack visualisation, developing new cache-level parallelism and prefetcher component metrics to analyse aspects of MLP which have been missing from literature and integrating these into a visualisation. We were also motivated by connecting MLP to performance, and identifying bottlenecks in MLP that can be targeted by both software and hardware optimisations. We have explored case studies using the metric on real programs and analysing the effect of both software and hardware changes. Our SPMV case study can be considered successful in demonstrating the utility of this stack by providing insight into caches, though perhaps not in the way we intended. Measuring DRAM MLP alone would have indicated that adding software prefetches decreases MLP, suggesting decreased performance. However, including L1 HCLP in the analysis shows that software prefetches in fact just moves the lost MLP at DRAM from core accesses up in the cache hierarchy to L1. Since L1 is much faster, this in fact improves performance. Our analysis also showed that MLP analysis exposed some trends which were not possible to observe with cache miss/hit rate metrics alone. Thus, Motivation 2, including cache behavior, has been validated partly here.

However, our GAPBS study was not as fruitful as we wanted. We had expected speculative MLP, in particular the ST bound component, to provide a guide for optimisation by indicating that more processor resources would help expose MLP, but our experiments did not justify this. Additionally, GAPBS itself did not demonstrate much core MLP, which meant we were not able to test the effect of the L1 MSHR as an MLP bottleneck in much detail. The L2 MSHR was unfortunately not explored due to lack of time either. We were able to observe the additional MLP injected by hardware prefetchers, validating their usefulness and addressing Motivation 3, but otherwise did not get enough out of this case study. It is likely that part of the reason for the low core MLP in GAPBS is that the benchmarks were designed to run multithreaded, so the base level parallelism was limited when running with a single core.

8.1 Limitations of our design

In the process of our case studies we discovered serious limitations with our design which suggested that some of the choices we had made were not ideal. These issues were unfortunately discovered late in the project because of two main reasons: (1) for much of the project we had only been working with total CLP and did not have as clear of an idea of the motivations of studying CLP as we do now, and some of the anomalous behaviours became apparent after splitting total CLP into miss and hit CLP, and (2) our data analysis pipeline was optimised to produce and analyse the graphs, but we were over-focused on the graphs and sometimes failed to look at the raw numbers behind them, which led to mis-interpretations of the data which ultimately cost us time. As a result, we did not have time to address or explore these limitations in detail. However, we hope that by discussing these problems, future researchers can learn and improve on our design.

The primary limitation of our design is that the denominator we chose to average over when computing average MLP was not well considered enough, and led to unintuitive effects. Ideally, the HCLP and MCLP should only depend on the pattern and timing of accesses to the specific cache level we are measuring at, and two benchmarks where the accesses to that cache level don't change should have the same HCLP and MCLP. However, we observed in the DRAM B/W and LLC size results in GAPBS where increased number of DRAM accesses and/or increased DRAM latency caused time spent memory bound to increase, which caused CLP metrics to change against our expectations. We had chosen our denominator with the goal of making the CLP at different levels more comparable, but the results indicate that the consequence was that changes in the denominator did not necessarily reflect the behaviors which we want to measure. One alternative choice is using the number of cycles where there at least one hit to level L for HCLP, and the number of cycles where there is at least one miss to level L for MCLP, which would at least ensure that the denominator is tied strictly to the behavior at level L of the memory hierarchy. Given time, we would have experimented further with different denominators to figure out which one gives the most reasonable behavior. At least it can be said that our denominator choice did not affect our SPMV results, as we verified in that case that the alternative denominators were in fact the same since there was practically always an ongoing access to DRAM, hit at L1 or miss at L1, so we were always just averaging over total runtime.

Another limitation was the fact that visual overlaps in the graph did not necessarily reflect actual overlaps. Our graph design intentionally involved bars overlapping with each other and hiding parts of each other. For example, the memory components' bars have total height equal to the total time spent active, while their vertical positions are such that the component of this total time which does not actually contribute to CPI, i.e. is not seen by the processor, is overlapped and hidden by other bars. This reflects the fact that the time spent executing itself overlapped. However, this design had two consequences. Firstly, the overlapping was designed to make sense on the vertical (CPI)

component of the graphs, but not on the horizontal (MLP/CLP) component, which caused important MLP/CLP information to be obscured and lost. For SPMV we had to use the total CLP graphs instead of hit CLP because the L1 hit CLP was in fact not visible due to being overlapped entirely by the DRAM block. Secondly, parts of bars that should have been overlapped actually were not. In the SPMV benchmarks, we actually initially misinterpreted the total runtime of the program. In a traditional CPI stack, the top of the stack is the total CPI or runtime, and hence we were reading the topmost height in the MLP stack as total runtime. However, in our graphs the total runtime is actually given by the top of the compute component's bar. We did not expect any bars to extend beyond the top of this height because the height from the bottom to the top of each bar is the total time spent active by that component which cannot be greater than the total runtime. However, the height from the x axis to the bottom of the bar is not 0, so the total height to the top of the bar can in fact exceed total runtime, which we observed with the DRAM component in the SPMV benchmark which had total active time equal to basically the entire runtime but which was positioned up a little bit because of the L1 component below it. This is depicted in Figure 8.1. This indicates that the overlapping design on the vertical component was flawed, and additionally that while the height to the bottom of each bar and the height within each bar made sense, the height to the top of the bar did not. For SPMV, the consequence was that we thought that adding software prefetches was actually hurting the performance of the program, which we failed to understand for a long time.

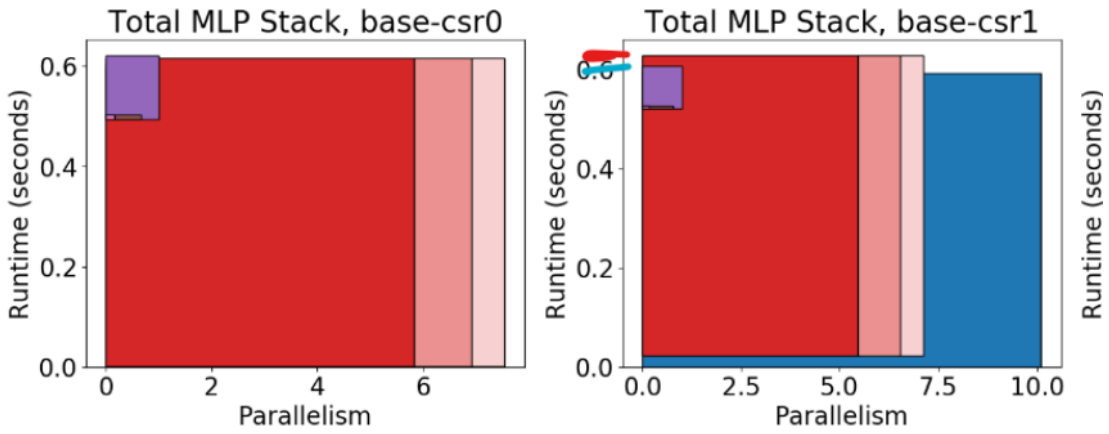


Figure 8.1: Figure showing the SPMV total MLP stacks before turning on prefetches (left) and after (right). On the right side graph, the blue line drawn shows the height representing the actual total runtime of the program, which is the top of the compute component. The red line drawn shows the actual top of the stack in this graph, which is in fact not meaningful at all but can easily be misinterpreted as the total runtime.

A relatively minor limitation is that in the SPMV case study, we found that it was made much more difficult to analyse because while we had isolated the MLP from hardware

prefetches, we had not done a similar thing for software prefetches. Ideally software prefetch MLP would also be split.

8.2 Future work

In addition to rectifying the limitations just discussed, which would likely be the main focus of immediate future work, the MLP stack requires more case studies to validate its usefulness. We would likely begin from conducting some synthetic benchmark studies, where we would write custom programs such as pointer chasing or maximum parallel memory accesses with expected MLP behaviour which we can verify. This would also make sure that we had the basics of the visualisation, such as the denominator, correct before proceeding. We also did not adequately explore the effect of MSHRs as we wanted to address Motivation 2b, and likely should have experimented with L2 MSHR size.

More radical future work would involve extending the MLP accounting scheme to work with multiple cores. We may also implement an instantaneous or timesliced view of MLP that can show CLP/MLP over time. This may help in addressing the issues caused by the denominator when computing average MLP/CLP because there is no need for a denominator when viewing MLP instantaneously. Implementing this can possibly be done easily with the existing infrastructure by keeping some kind of MLP trace in the accounting code. The main benefit of timesliced MLP however is being able to view phase behavior in a program and see MLP changing over time.

Bibliography

Beamer, S.; Asanović, K.; and Patterson, D., 2015. The gap benchmark suite. (Aug. 2015). doi:10.48550/ARXIV.1508.03619. [Cited on pages 32 and 34.]

Beamer, S.; Zhang, Y.; Sutton, M.; Fidel, A.; Yoon, W.; Shiming; Janda, K.; Ahn, J.; and Masterton, C. Gapbs benchmark suite (source code). Online. https://github.com/sbeamer/gapbs. [Cited on page 34.]

Bois, K. D.; Eyerman, S.; Sartor, J. B.; and Eeckhout, L., 2013. Criticality stacks: Identifying critical threads in parallel programs using synchronization behavior. *Proceedings of the 40th Annual International Symposium on Computer Architecture*, (2013), 511–522. doi:10.1145/2485922.2485966. [Cited on page 11.]

Carlson, T. E.; Heirman, W.; Allam, O.; Kaxiras, S.; and Eeckhout, L., 2015a. The load slice core microarchitecture. In *Proceedings of the 42nd Annual International Symposium on Computer Architecture*, ISCA '15 (Portland, Oregon, Jun. 2015), 272–284. Association for Computing Machinery, New York, NY, USA. doi:10.1145/2749469.2750407. [Cited on page 2.]

Carlson, T. E.; Heirman, W.; and Eeckhout, L., 2011. Sniper: exploring the level of abstraction for scalable and accurate parallel multi-core simulation. *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*, (2011). doi:10.1145/2063384.2063454. [Cited on page 25.]

Carlson, T. E.; Nilakantan, S.; Hempstead, M.; and Heirman, W., 2015b. Epoch profiles: Microarchitecture-based application analysis and optimization. *IEEE Computer Architecture Letters*, 14, 1 (2015), 30–33. doi:10.1109/lca.2014.2329873. [Cited on pages 18 and 19.]

Chou, Y.; Fahs, B.; and Abraham, S., 2004. Microarchitecture optimizations for exploiting memory-level parallelism. *Proceedings. 31st Annual International Symposium on Computer Architecture, 2004.*, (2004), 76–87. doi:10.1109/isca.2004.1310765. [Cited on pages 2, 5, 8, 13, 18, and 19.]

Eyerman, S.; Bois, K. D.; and Eeckhout, L., 2012. Speedup stacks: Identifying scaling bottlenecks in multi-threaded applications. *2012 IEEE International Symposium on Performance Analysis of Systems & Software*, (2012), 145–155. doi:10.1109/ispass.2012.6189221. [Cited on page 11.]

Eyerman, S.; Eeckhout, L.; Karkhanis, T.; and Smith, J. E., 2006. A performance counter architecture for computing accurate cpi components. *ACM SIGARCH Computer Architecture News*, 34, 5 (2006), 175–184. doi:10.1145/1168919.1168880. [Cited on pages 10 and 12.]

Intel, 2009. Intel® xeon® processor x5550. Online. http://ark.intel.com/products/37106. [Cited on page 31.]

Mehta, S., 2022. Performance analysis and optimization with little's law. 12–23. IEEE, Singapore, Singapore. doi:10.1109/ISPASS55109.2022.00002. Literaturangaben. [Cited on pages 2, 3, 4, and 15.]

Tang, X.; Kandemir, M. T.; Karakoy, M.; and Arunachalam, M., 2019. Co-optimizing memory-level parallelism and cache-level parallelism. *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, (2019), 935–949. doi:10.1145/3314221.3314599. [Cited on pages 3, 11, and 17.]

Zeitak, A. and Morrison, A., 2021. Cuckoo trie. *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, (2021). doi:10.1145/3477132.3483551. [Cited on page 2.]